Spring Boot - Selected elements

Cookies, session, security (users, roles), API controller

Syllabus

- Cookies
- Session
  - CSRF (Cross-Site RequestForgery) attack
- Spring Security:
  - Authentication and authorization
  - Filters in Spring Boot
  - Users/Roles
  - Role-based routing security
- API controller:
  - JSON format
  - ReST and RESTful approach
  - API controller for communicating with the frontend in an SPA application.
  - Postman - Application for testing.

Cookies

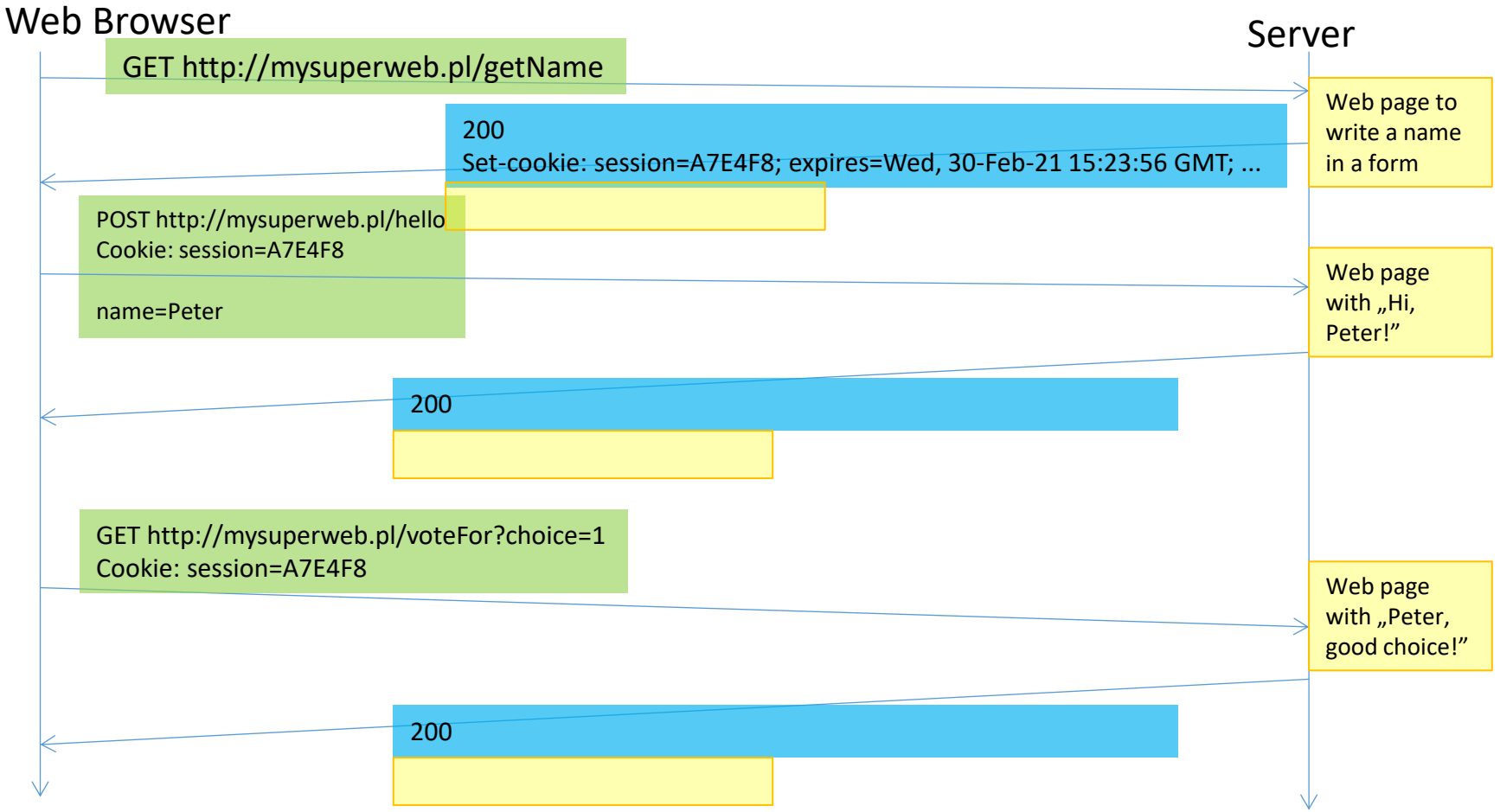
- HTTP is stateless
- The **need** to combine **consecutive requests** to remember that they are **from the same user**
- Mechanism – **cookies**
  - They are transmitted **in the header**: requests or responses
    - Invisible to a regular user
  - A cookie has a **key** (text) and a **value** (text)
  - They have a fixed **expiration date**, transmitted in **response** cookies:
    - point in time
    - without specifying the validity time – for the duration of the session, understood as the time of operation of the browser
  - The response cookie also has other features (or access only for the source domain, or for all domains, etc.)
- In cookies you can directly hold certain information from the user:
  - color preferences,
  - the order of blocks with information that user prefers
  - the shopping cart,
  - which ads the user clicked on (user tracking),
  - **a session**
  - etc.

Cookies and the browser

- The browser works according to a simple algorithm:
  - For cookies in response:
    - Remembers **all cookies** from the domain where the answer came from,
    - If the cookie with the given **key** was **already** there, it previously **deletes** the **previous** one.
- When sending a request to a specific domain, it sends **all unexpired** cookies (also from **another tab** or **another instance** of the same browser):
  - **Expired** cookies are **deleted** from the browser's memory.
- When the browser **finishes** (**starts**) it **deletes** all **session** and possibly **expired** cookies:
  - However, the rest **is saved** in the file until the next time you turn on the browser!
- Cookies should be **small** (one up to 4096 bytes) and their number for one domain is often limited (usually also up to 4KB).
- The browser **user** can **manipulate** cookies (**delete**) or even **prohibit** their use for a specific domain.

Cookies - session

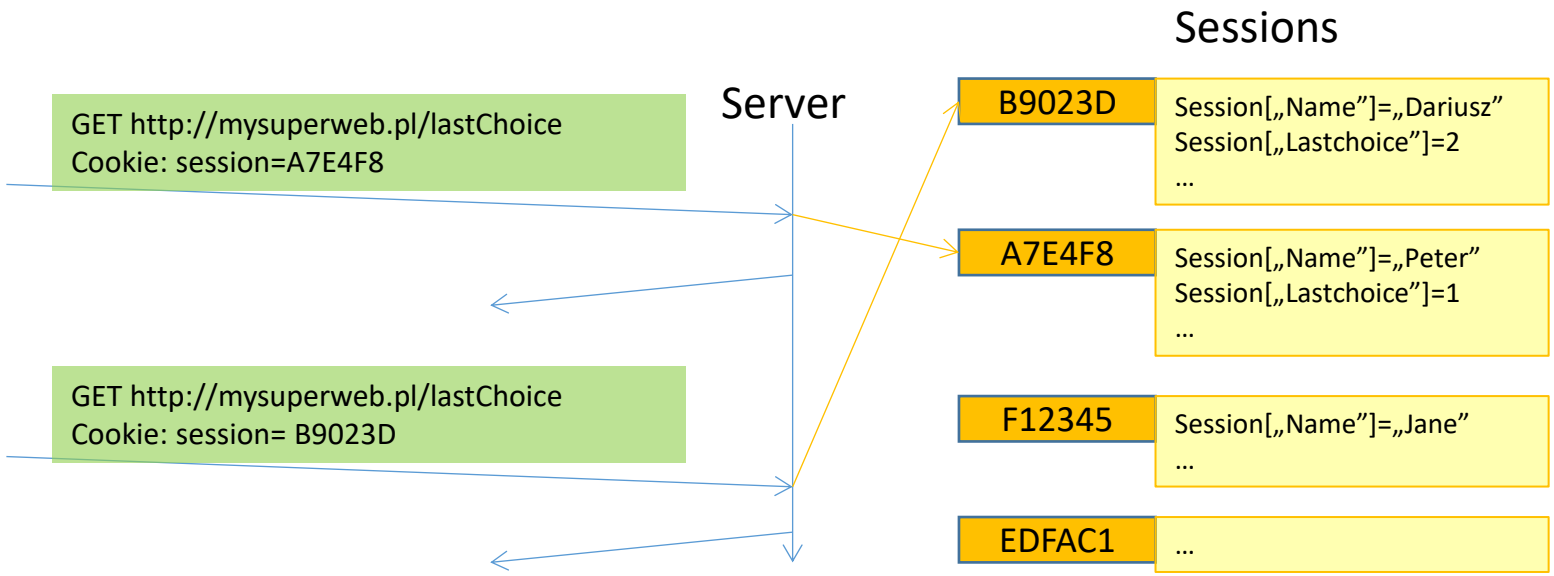
- The most commonly used cookies are to remember the session with the user and the data related to the session.
- A **session** is understood as **a sequence of consecutive requests** from the **same user**.



Session cookie – server operation

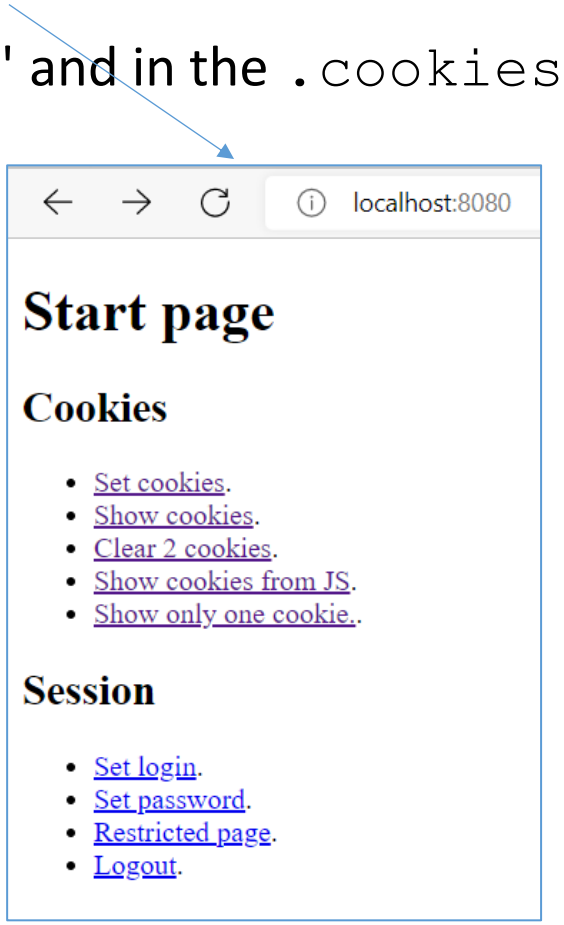
Server:

- Creates cookies and adds to the response to the request:
  - A new cookie or modification of a previously sent one (e.g. by specifying an expiry date that has already passed will delete the cookie from the browser's memory).
- It reads the cookie **from the request** and decides on its basis to take appropriate action.
- In the case of **sessions**, the values held in the cookie are a **random string**, for which the key-value **dictionary** is remembered on the server side (the session's cookies themselves are the keys of the **higher-level dictionary**):
  - The server reads the session cookie from the request, finds the dictionary associated with it and uses the keys of this dictionary to prepare the response page,
  - The server also remembers the cookie data (e.g. with an expiration date), for example, to free up memory for dictionaries no longer used.



Cookies in Spring

- Project `CookiesAndSession`
  - Dependencies:
    - Spring Web
    - Thymeleaf
    - Spring Session
  - `index.html` page **with links** to all controller actions
  - `CookieController` with request path prefix "`/cookies/`" and in the `.cookies` subpackage handling requests:
    - `set` – sets all cookies
    - `show` – showing all cookies
    - `clear2` – deletes the first two cookies
    - `showjs` – showing cookies **using JavaScript** code
    - `onlyone` – shows only the cookie `isactive`
  - Views in a subfolder `/cookies`
  - Cookies:
    - `name` – session's
    - `color` – for 7 days
    - `isactive` – for 10 seconds
    - `hiddenToken` – for 1 minute, not for JavaScript



Security issues

- Spring Expression Language (**SpEL**) - used by Thymeleaf to interpret expressions of the type `${...}`.
- Since Spring Framework 6 / Spring Boot 3, SpEL security has been tightened. Spring now blocks access to most types from the namespace:
  - `java.*`
  - `jakarta.*`
  - `sun.*`
  - and many others that are not Spring beans or model objects by default.
- This means that even if you pass a `Cookie[]` array to the model, Thymeleaf won't allow you to read the properties (name, value, etc.), because the `Cookie` comes from the `jakarta.servlet.http` package, which is outside the SpEL type whitelist.
- Possible solutions:
  - repack only the necessary data from the `Cookie` into a **map** (dictionary)
  - repack it into your **own simple type** - this solution will be used.

`PairNameValue` helper class

- A helper class for storing lists of name-value pairs.
- In most cases, an all-argument constructor and getters are sufficient.
- When developing applications, you often need to create simple classes like these.
  - Java doesn't easily support **tuples**, which would be useful in such cases.

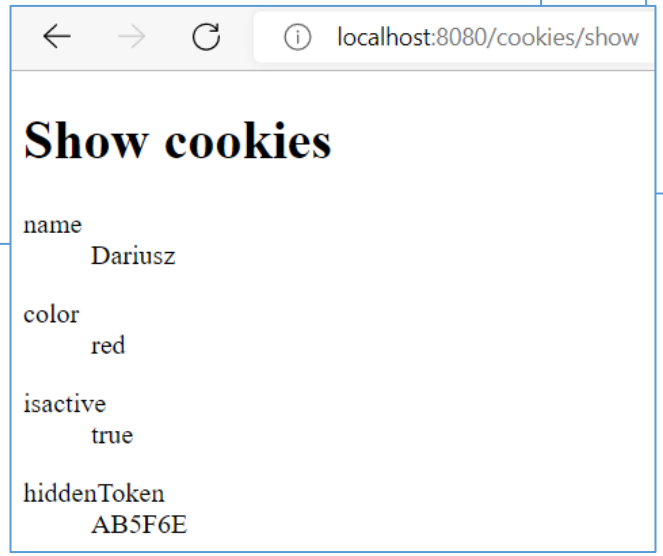
```java
@Getter
@AllArgsConstructor
public class PairNameValue {
    public String name;
    public String value;
}
```

Showing cookies

- Injecting an `HttpServletRequest` object with request data (other header elements can be read in addition to cookies) into the `showCookies()` action and executing the `getCookies()` method and repacking to a list of name-value pairs allows you to pass an array of cookies to a view
- In the view for each cookie, we download and show its name (via `name`) and value (via `value`)
- File fragment `CookiesController.java` and view `/cookies/show.html`.

```java
@Controller
@RequestMapping("/cookies/")
public class CookiesController {
        …
    @GetMapping("show")
    public String showCookies(HttpServletRequest request, Model model) {
        Cookie[] cookies = request.getCookies();
        List<PairNameValue> list=new ArrayList<PairNameValue>();
        for (Cookie cookie : cookies) {
            list.add(new PairNameValue(cookie.getName(), cookie.getValue()));
        }
        model.addAttribute("cookies",list);
        return "/cookies/show";
    }
```

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org en">
<head>
    <meta charset="UTF-8">
    <title>Show cookies</title>
</head>
<body>
<h1>Show cookies</h1>
<dl th:each="cookie : ${cookies}">
    <dt th:text="${cookie.name}"></dt>
    <dd th:text="${cookie.value}"></dd>
</dl>
</body>
</html>
```



Creating cookies

- Injecting an `HttpServletResponse` object into the `setCookies()` method allows you to add cookies to the response header using the `addCookie()` method.
- Cookies are objects of the `Cookie` class, in the constructor you must specify the **name** and **value** of the cookie. By default, it is a **session's** cookie (for the duration of the browser's operation, although disabling it does not delete such a cookie).
- After creation, you can use the `setMaxAge()` method to specify **the expiry date** of the cookie in **seconds**.
- You can also use the `setHttpOnly()` method to indicate that the cookie is **only for** queries from the **browser** (it will **not be available** from JavaScript).

```
public class CookiesController {
    @GetMapping("set")
    public String setCookies(HttpServletResponse response) {
        Cookie browserSessionCookie = new Cookie("name", "Dariusz");
        response.addCookie(browserSessionCookie);
        Cookie weekCookie = new Cookie("color", "red");
        weekCookie.setMaxAge(7 * 24 * 60 * 60); // expires in 7 days
        response.addCookie(weekCookie);
        Cookie fastCookie = new Cookie("isactive", "true");
        fastCookie.setMaxAge(10); // expires in 10 seconds
        response.addCookie(fastCookie);
        Cookie notForJsCookie = new Cookie("hiddenToken", "AB5F6E");
        notForJsCookie.setMaxAge(60); // expires in 1 minute
        notForJsCookie.setHttpOnly(true); // not for JS
        response.addCookie(notForJsCookie);
        return "redirect:/cookies/show";
    }
            …
```

Deleting a cookie

- To delete a cookie, send a cookie with the same name, but **over time** the value **in the past**, that is, run `setMaxAge(0)` for it.
  - Below is the execution of this for the **first two cookies**.

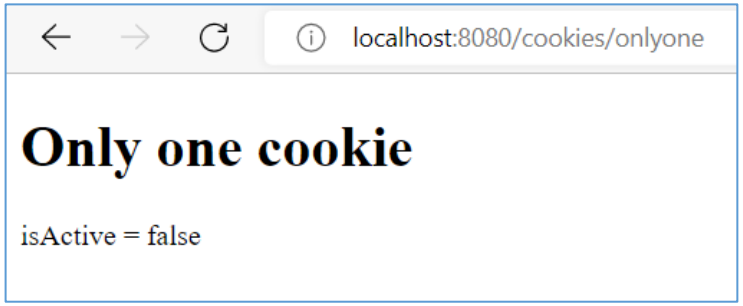
```
    @GetMapping("clear2")
    public String showCookies(HttpServletResponse response, Model model) {
        Cookie browserSessionCookie = new Cookie("name", "Dariusz");
        browserSessionCookie.setMaxAge(0); // clear cookie
        response.addCookie(browserSessionCookie);
        Cookie weekCookie = new Cookie("color", "red");
        weekCookie.setMaxAge(0); // clear cookie
        response.addCookie(weekCookie);
        return "redirect:/cookies/show";
    }
```

Injecting a cookie

- If only a few specific cookies are required for a given action, they can be **injected** as method parameters by annotating `@CookieValue()` with the `name` attribute of the cookie, and as a `default` the value that will be assigned if such a cookie is **not present** in the request.
- You need a new view only for the selected cookie (in the `onlyone.html` file)

```
@GetMapping("onlyone")
public String getCookieInParams(@CookieValue(name = "isactive", defaultValue = "false") String isActive,
                                Model model) {
    model.addAttribute("isActive",isActive);
    return "/cookies/onlyone";
}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Only one cookie</title>
</head>
<body>
<h1>Only one cookie</h1>
<p>
    isActive = <span th:text="${isActive}"></span>
</p>
</body>
</html>
```



Cookie not available from JavaScript

- To see which cookies are available from JavaScript, get them in the script using `document.cookie`. They are in the form of a string that the browser sends during queries
  - View in the `showjs.html` file
  - The `showCookiesFromJs()` action in the controller only directs to this view.
  - As you can see, the `hiddenToken` cookie is not presented

```
@GetMapping("showjs")
public String showCookiesFromJs() {
    return "/cookies/showjs";
}
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Show cookies from JavaScript</title>
</head>
<body>
<h1>Show cookies from JavaScript</h1>
<p id="cookies">

</p>
<script>
    var p=document.getElementById("cookies");
    p.innerText=document.cookie;
</script>
</body>
</html>
```

Other information about cookies

- TODO: testing the application in different sequences of actions.
- The `setSecure(true)` method allows you to send a cookie **only** over a secure HTTP**S** connection
- There are **several properties** that can be set for cookies
- There is a `WebUtils` class and a `ServletRequestUtils` class that makes it easier to manipulate cookies and other data related to request or response headers.
  - For example, to find a cookie with a given name.

Session

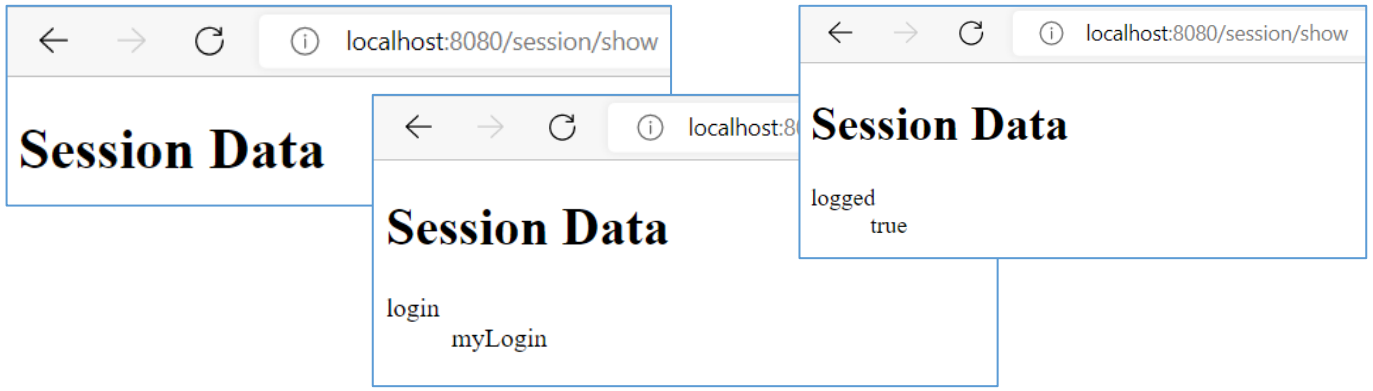
- Project `CookiesAndSession` – continuation
  - A `SessionController` with a request path prefix of "`/session/`" and a `.session` subpackage that handles requests:
    - `pagelogin` – to enter your login
    - `pagepassword` – to enter the password, if a login has been previously provided. If not, redirect to `pagelogin`. If the user has provided the correct data – redirection to `restricted`. If not - to the main page of the application.
    - `restricted` – The page is available only to **logged** in users (for simplicity, only the `root` user with the password `toor` is available). For incorrect data, it returns to `pagelogin`.
    - `pagelogout` – logout, deletes session data on the server, returns to `pagelogin`.
  - Views in a subfolder `/session`
  - Session data:
    - `login` – remembers what login was given on the `pagelogin` page
    - `isLogged` – set to `true` when correct data is provided, removes `login` from session data
- Inject an `HttpSession` object to manipulate session data by using methods:
  - `setAttribute(<name>, <value>)` – Set an attribute `<name>` with a value `<value>`
  - `getAttribute(<name>)` – Get attribute `<name>` values
  - `getAttributeNames()` – returns a list of strings with the names of **all attributes**

Session controller – action and `show` view

- To shorten the record, the collection was replaced with a **stream**, which used the **mapping from** the name of the session attribute **to** the name and value **pair**, which was passed to the view using the model (which shows it as a list of definisions)
- For this purpose, the previously created `PairNameValue` class will be used.

```java
@Controller
@RequestMapping("/session/")
public class SessionController {
        …
    @GetMapping("show")
    public String show(HttpSession session, Model model) {
        var sessionElems= Collections.list(session.getAttributeNames()).stream()
                .map(name -> new PairNameValue(name,session.getAttribute(name).toString()))
                .toList();
        model.addAttribute("sessionElems",sessionElems);
        return "/session/show";
    }
}
```

```java
@Getter
@AllArgsConstructor
public class PairNameValue {
    public String name;
    public String value;
}
```




```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org en">
<head>
    <meta charset="UTF-8">
    <title>Session Data</title>
</head>
<body>
<h1>Session Data</h1>
<dl th:each="elem : ${sessionElems}">
    <dt th:text="${elem.getName()}"></dt>
    <dd th:text="${elem.getValue()}"></dd>
</dl>
</body>
</html>
```

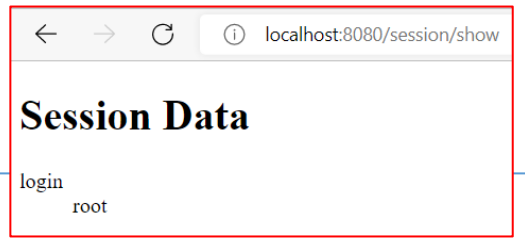
Action and `pagelogin` view

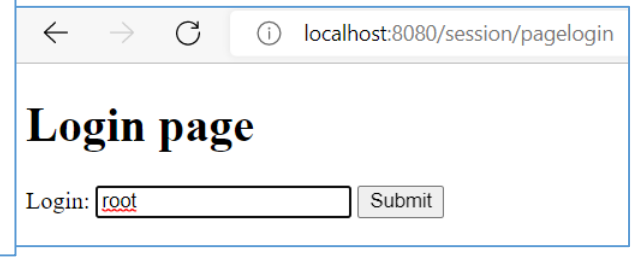
- Two actions: `login` (for GET request) and `loginPost` (for POST) and `pagelogin` view. The second action (with the `HttpSession` object injected) sets the session to the "`login`" attribute with the value received from the request and redirects to the password entry page.
  - For security reasons, it removes the "`logged`" attribute.
- Through the previously described method, you can check the presence of this attribute in the session.

```
public class SessionController {
            …
    @GetMapping("pagelogin")
    public String login(Model model) {
        String login= "";
        model.addAttribute("login",login);
        return "/session/pagelogin";
    }

    @PostMapping("pagelogin")
    public String loginPost(@RequestParam("login") String login, HttpSession session) {
        session.setAttribute("login",login);
        session.removeAttribute("logged");
        return "redirect:/session/pagepassword";
    }
}
```

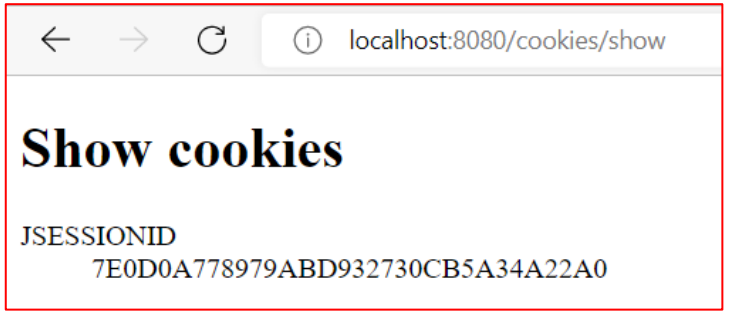
```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org en">
<head>
    <meta charset="UTF-8">
    <title>Login</title>
</head>
<body>
<h1>Login page</h1>
<form th:action="@{/session/pagelogin}" method="post">
    <label >Login:   </label>
    <input type="text" th:value="${login}" name="login" autofocus>
    <input type="submit" value="Submit">
</form>
</body>
</html>
```

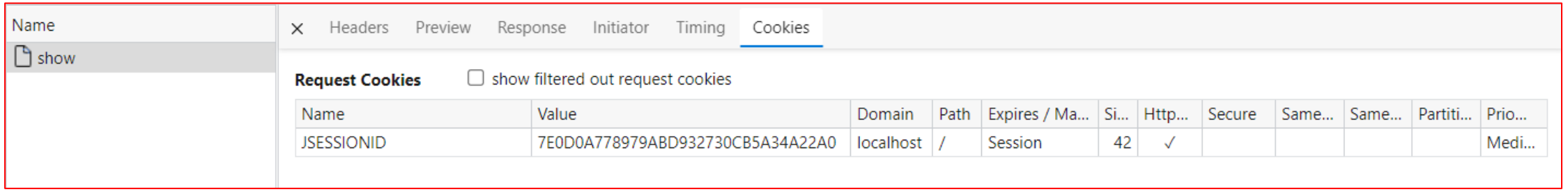




Session cookie

- Thanks to the previous part for the demonstration of how cookies work, you can check that the `JSESSIONID` cookie has been set.
- It is not available for JavaSciptu.
- Of course, this cookie can also be found in browser tools.





Password typing support

- The `pagepassword` view is very similar to the `pagelogin` view. (hence the lack on the slide)
- The action for GET checks whether the login has been entered earlier (if not, it redirects to entering the login).
- The action for POST also checks whether the login was provided. He compares it with the word "`root`" and the password with the word "`toor`". If everything is correct, it removes the "`login`" attribute from the session and sets the "`logged`" attribute to **`true`**, redirecting to the `restricted` page.

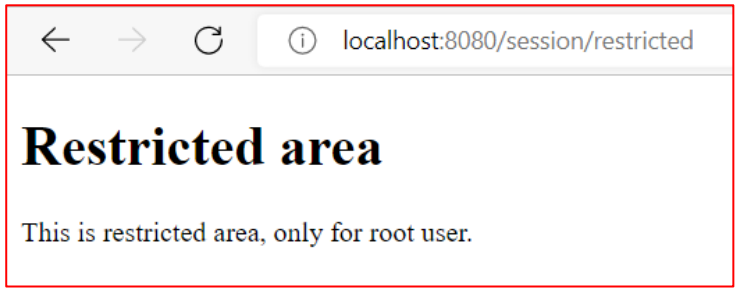
```
@GetMapping("pagepassword")
public String password(HttpSession session, Model model) {
    String login= (String) session.getAttribute("login");
    if(login==null)
        return "redirect:/session/pagelogin";
    String password="";
    model.addAttribute("password",password);
    return "/session/pagepassword";
}
@PostMapping("pagepassword")
public String passwordPost(@RequestParam("password") String password, HttpSession session) {
    String login= (String) session.getAttribute("login");
    if(login==null)
        return "redirect:/session/pagelogin";
    if(login.equals("root") && password.equals("toor")) {
        session.removeAttribute("login");
        session.setAttribute("logged", true);
    }
    return "redirect:/session/restricted";
}
```

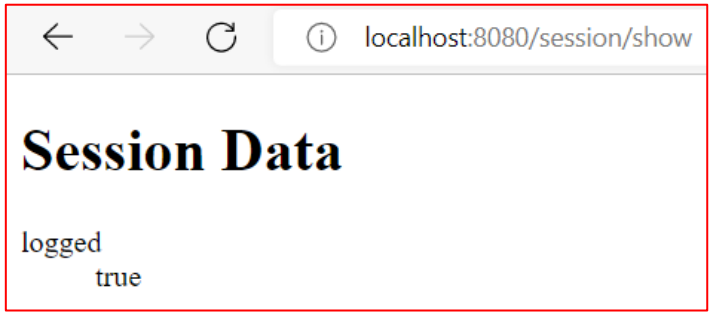
Site for logged-in users

- The restricted page can only be accessed if the session is set to the "`logged`" attribute.
- You can enter it repeatedly.
- On the session data page, you can only see this "`logged`" attribute.

```
    @GetMapping("restricted")
    public String restricted(HttpSession session) {
        System.out.println("jestem 1");
        Boolean logged= (Boolean) session.getAttribute("logged");
        if(logged==null)
            return "redirect:/session/pagelogin";
        return "/session/restricted";
    }
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Restricted area</title>
</head>
<body>
<h1>Restricted area</h1>
<p>This is restricted area, only for root user.</p>
</body>
</html>
```





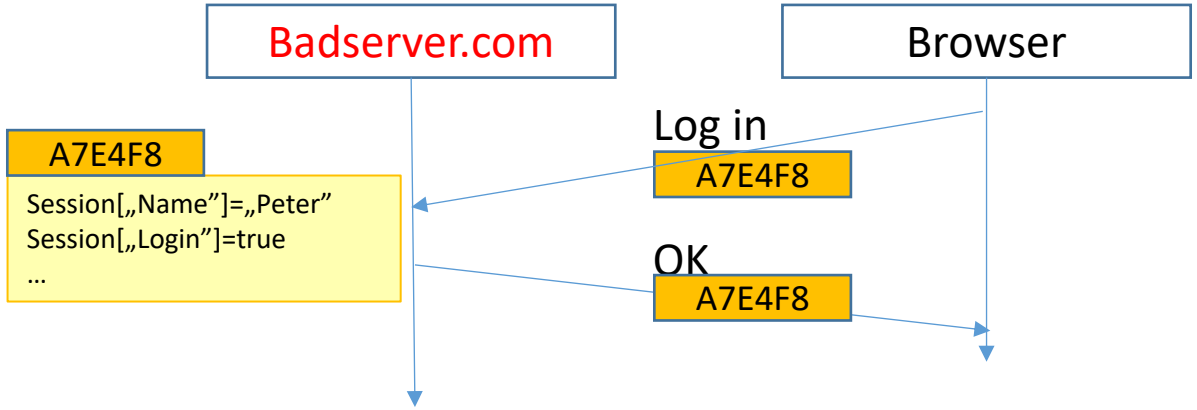
Sign out

- To implement logout, simply remove the "`logged`" attribute in the session.

```
@GetMapping("logout")
public String logout(HttpSession session) {
    session.removeAttribute("logged");
    return "redirect:/";
}
```

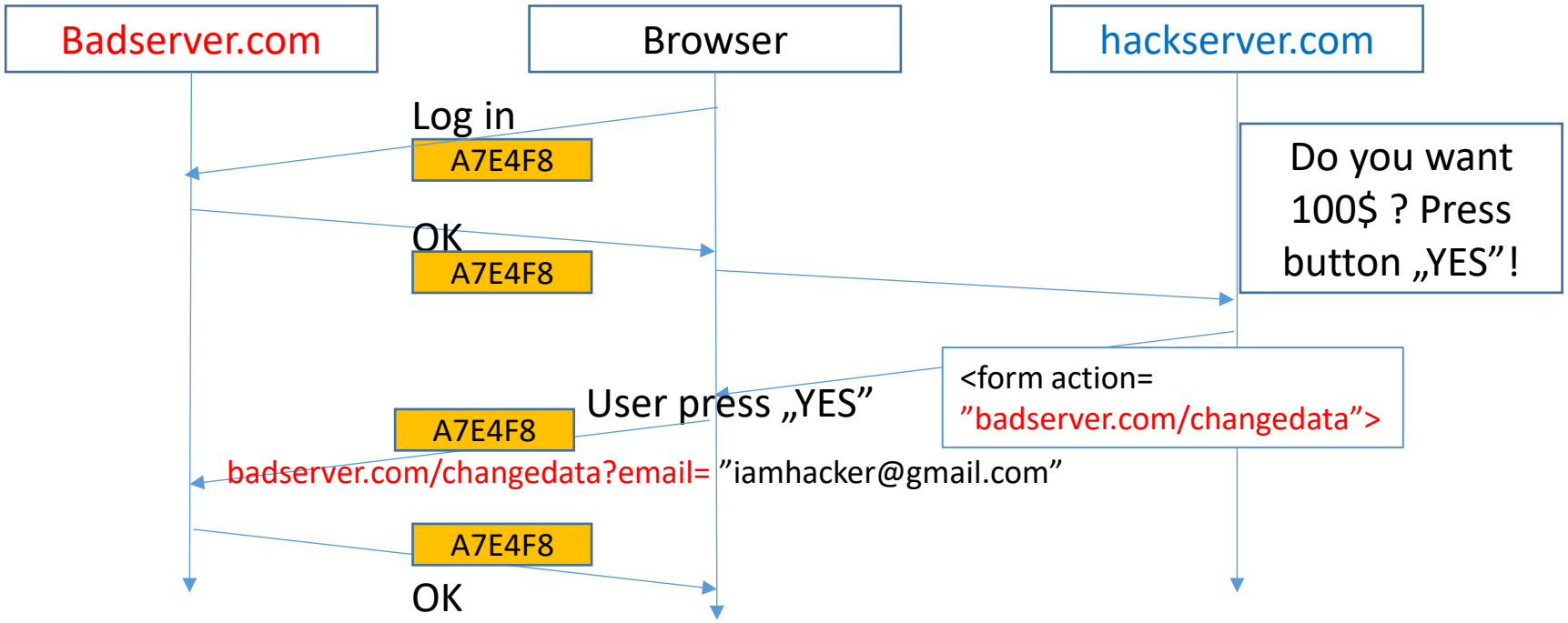
CSRF (Cross-Site Request Forgery) 1/3

- Cross-site request – request to a different domain than the page from which we want to make it (e.g. a simple link)
- The cookie mechanism **enables** a CSRF attack. If we **do not protect** form pages against it, the following scenario is possible (e.g. on the badserver.com domain).
- We want an unaware user to modify **their** data, e.g. email address, which is often used to **recover** passwords and log in. Changing the address involves filling out a form (page sent by a GET request) on the `badserver.com/changedata` page and sending it to the same address (but by POST request), but of course only if the session variable points to the correct user. Knowing all this, an attack was prepared on the `hackserver.com`.
  - The user logs in to his account on badserver.com
  - The browser remembers the session variable and which user is logged in.



CSRF (Cross-Site Request Forgery) 2/3

- **At the same time, in the same browser**, the user opens a `hackserver.com` page, prepared to attack his account.
- There is a simple form with a button "Do you want to win $ 100?", but in the **hidden** fields it has the parameters required for the form to change data by the `badserver.com/changedata` page (e.g. the "`email`" field is set to "`iamhacker@gmail.com`") and there redirects sending the form (**cross-site request**).
- The browser sees that there will be a request to the `badserver.com domain`, so it adds **all cookies** for that domain in the header, **including the session cookies**.
- The change of data on the server `badserver.com` **will be made**, because the **cookies match**!
- The user may see information about the change of data (or nothing at all) and return surprised to the domain `badserver.com`.



CSRF (Cross-Site Request Forgery) 3/3

- The creator of `hackserver.com` does not know if the attack was successful, but from time to time he tries to "recover" the password to the account `iamhacker@gmail.com`

- Security method:
  - For each form sent by GET, adding a **hidden** field (**token**), e.g. `antiForgery` with a random value, which **must match** when the data from the form **returns** with a POST request,
    - To remember the value of the token, you can use a cookie (**not available** from Java),
    - The above attack will fail, because the card/JavaScipt has access only to its card data and cookies (properly configured) **from the original domain** of the website.
- CSRF attack has many variations: using a Java script, with a crafted image, etc. In any case, this security shall be sufficient.

Other selected informations about cookies

- Property `samesite`:
  - `lax` : allows you to block most CSRF attacks, while for simple links (e.a. `href="http://mysuperweb.com/Student/Create"`), in documents on non-domain pages, cookies will be sent by the browser
  - `strict`: even for simple links (as for the `lax` option), **no cookies** will be included by the browser for **cross-site requests**.
  - `none`: no restrictions on sending cookies for cross-site requests
- Not all details about cookies or their use have been presented.
- In **European law**, the **user must consent** to the **storage** of cookies for our website.
- Browser configuration may cause the **session** cookie not to always be deleted when you close the browser (Firefox).

Project `CookiesAndSession` - remarks

- The example is a simplification – user data is permanently entered in the application.
  - User data should be an external database.
- It would be useful to simplify checking which action can be run by a given user.
  - It is best to enter roles such as `admin`, `teacher`, `student`, etc. or permissions e.g. `CanWrite`, `CanEnroll`, etc.
  - Instead of writing a conditional expression in your code first (e.g. `if (user.role=="teacher"))` you should only be able to insert an **annotation** (or **other configuration** mechanism) that this is an action for a given role.
- In Spring Boot, such a security management module is, for example, **Spring Security**, which has, among others, the **functionalities described above**.

Spring Security

Authentication and authorization (by role)

Authentication, authorization

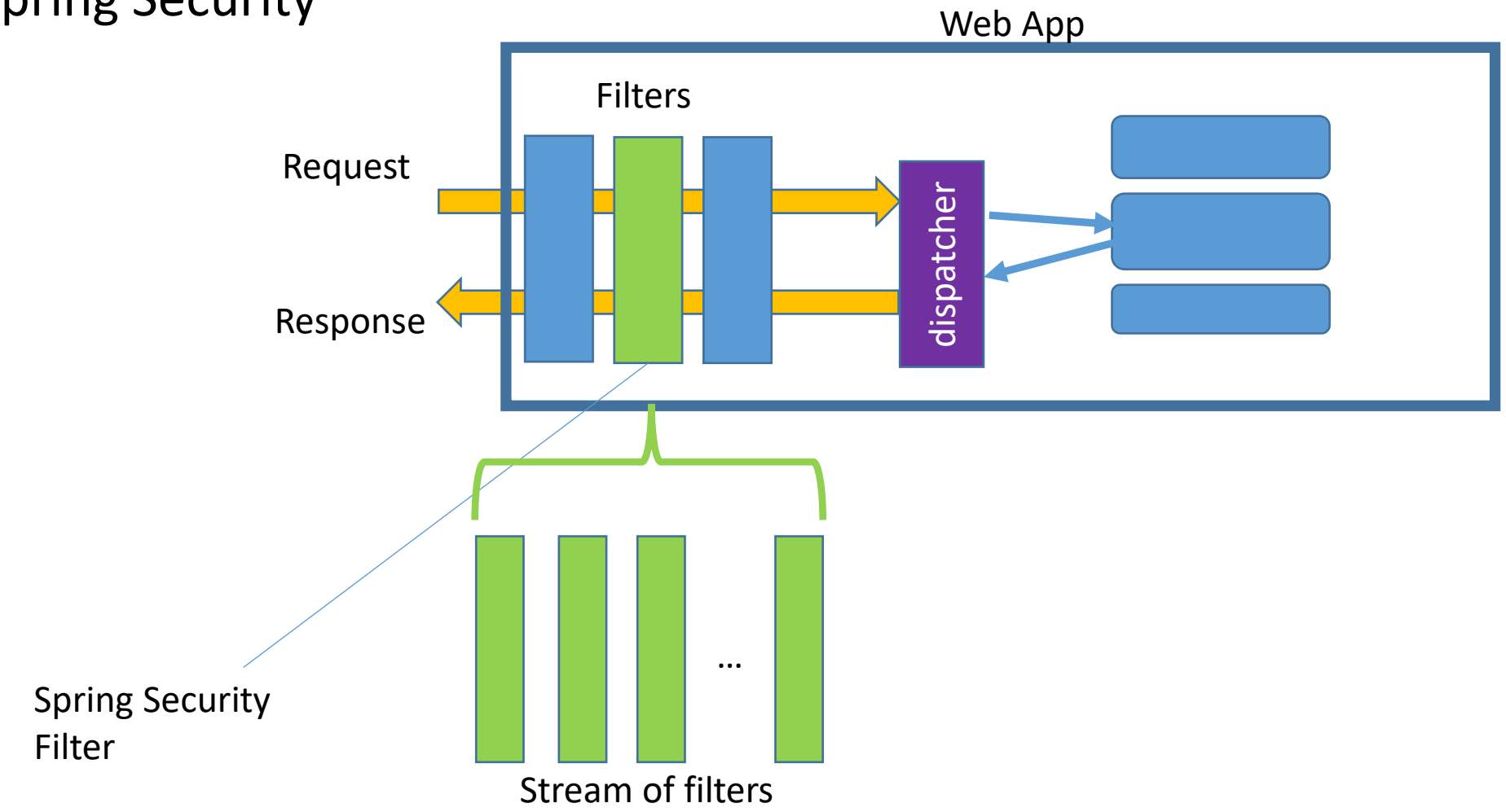
- **Authentication:** This is to verify that the user is who they say they are. So the problems here are "`Who is this?`" and "`How do I know it's this person?`"
  - Most often by entering a **login** and **password**, because knowledge of the password indicates that the user is who he claims to be
  - Other solution: authentication by a **trusted third party** (e.g. Facebook, Google, etc.)
- **Authorization:** This is the verification that the user **has the right** to access specific services/resources. The questions are "**Can user X read Y?**", "**Can user X change Z?**"
  - The simplest solution is to give the **role** (or **roles**) to each user. A user with a given role can use the selected services.
  - Another solution: **privileges** assigned to roles, **credential system**, etc.

Spring Security

- Solutions for authentication and authorization are combined into one "module".
- Since there are **many** authentication and authorization mechanisms (and their combinations with each other), the module dealing with these programming elements is more complicated, but it must also be very flexible.
  - Where we store user data (memory, MySQL, ...)
  - What authorization mechanism (roles, privileges, credentials, ...)
  - How do we encrypt passwords,
  - etc.
- Spring Security – huge possibilities of configuring authentication and authorization mechanisms

Spring Security - architecture

- The place of filters in the Spring Boot architecture.
- Filtr Spring Security

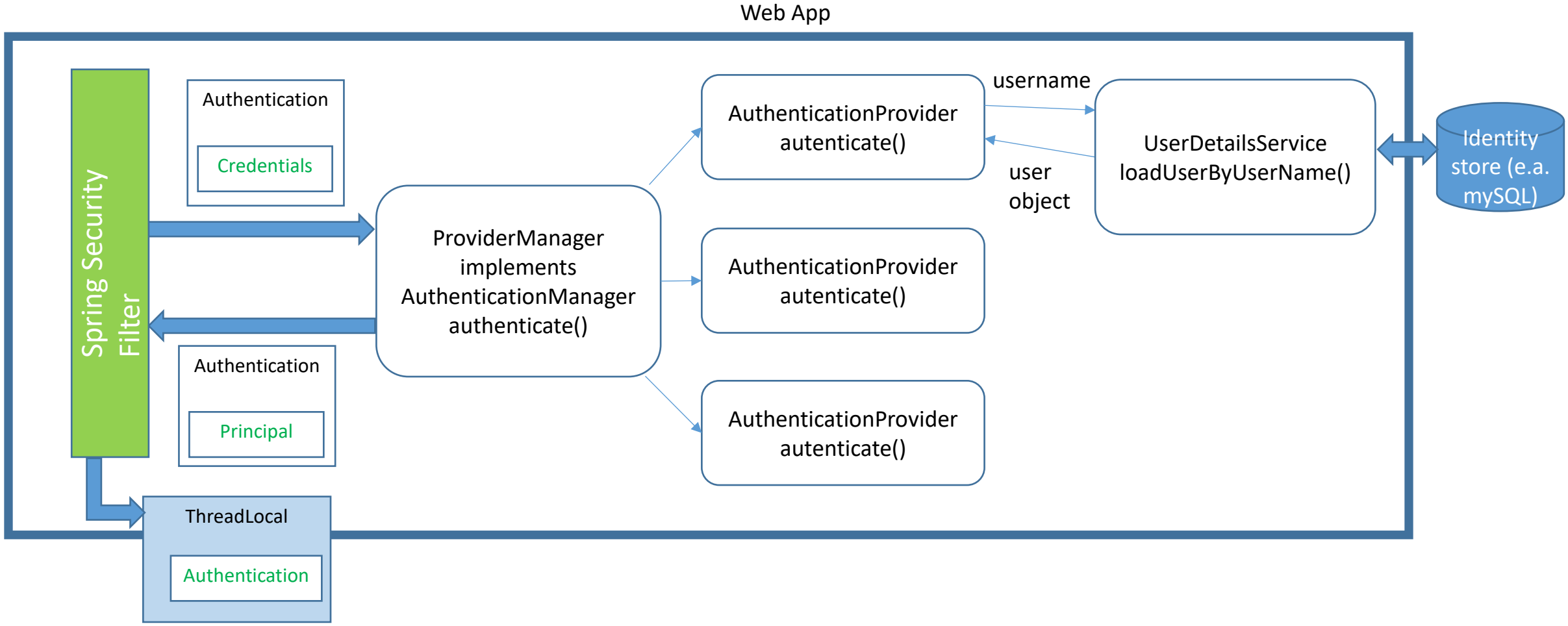


Spring Security – architecture - operation

- The **request** (character stream) from the user (browser), before it reaches the servlet controller, is converted into an object with pre-processed request components (request line, headers, request body). There are also request response elements in this object (also partially already populated).
- Then such an object passes through the so-called **filters**.
- Filters are **combined into a string** (**stream**).
- A filter is a class that receives a request processed in previous filters (in the form of an object) as well as possible other data (data collection), e.g. session variables.
  - You can create additional data needed in the next filter, which means that the filters must be properly organized.
- After generating a response in the servlet, it passes through filters **in reverse order** (returns) and the response can also be processed.
- In addition, the filter may decide to generate a response immediately, i.e. the processing **does not reach** subsequent filters and controllers.
  - For example, when a user doesn't have permission to a page, an error page is generated.
- The **Spring Security filter** is responsible for determining whether a user is logged in, what permissions they have, and whether these permissions allow them to handle a given request.
  - Basically, it's a substring of filters.

Authentication and authorization process

- Diagram of the Spring Security filter.



Authentication and authorization process – action 1/2

- The operation of the Spring Security Filter (**SprSec**) is complicated, because it must be prepared in various ways of authentication and authorization. To make this possible, many classes are needed for different purposes.
- At the beginning of operation, **SprSec** checks whether the session variables indicate that the user **is logged** in and what role (or permissions/credentials) they have.
  - In the `ThreadLocal` thread.
- If this data **is** in session variables, it uses it to determine whether a given request can be handled for that user.
  - If **yes** – forwards to the **dispatcher**
  - If **not** – returns a page with an access error
- If the request contains **login information**, it replaces it with an `Authentication` object containing **credentials**. In the **simplest** case, it is a **login** and **password**. This object is passed to the `ProviderManager` (which implements the `AuthenticationManager`, specifically the `authenticate()` method).
- This is the object responsible for informing whether the login is correct and what permissions the logged-in user has. The currently logged in user with all information about him (including permissions) is referred to as a `Principal` class object (principal).

Authentication and authorization process – action 2/2

- To do this, the `AuthenticationManager` selects the correct `AuthenticationProvider` (depending on **the type of authentication**) to perform a check by `authenticate()` whether the authentication was successful and what permissions the principal will have.
- The `AuthenticationProvider` object must be able to access the data it needs to analyze to perform the task. The data can be in multiple places, so it uses a `UserDetailsService` object that implements the `loadUserByUserName()` method. The method requires a **user name** and returns an **object** that **describes** the user (e.g., **password**, **roles**, etc.)
- The `AuthenticationProvider` object parses credential data with user data and sets the `Principal` data, which it returns to the `AuthenticationManager`, which passes it to the **SprSec** filter. The **SprSec** filter will store this data (associate it with the session cookie) in order to check what permissions it has **in subsequent requests** from the same user (`ThreadLocal` element in the figure).
- `UserDetailsService` class objects may need database access classes (JPA, ODBC) to function, but they may also have data in memory (test approach).

Authentication and authorization process - configuration

- For the previously described process to work correctly, **many classes** are needed.
- In Spring Security, everything has been prepared in such a way that through **other classes** (and their methods) **only configure** what solutions have been adopted in the web application (and not only the web application) to make everything work.
  - Therefore, you do not directly create classes/objects `ProviderManager`, `AuthenticationProvider`
  - But sometimes you have to implement the `loadUserByUserName()` method in the appropriate class
- In addition, it is possible to **add restrictions on access** to specific controller actions (**expressed by the URL**) for selected types of users (**authorization mechanism**).
  - for example, you can add a rule that all actions triggered by a URL with the prefix "`/adm`" will be only possible for a user with the "`ADMIN`" role.
- Since the number of combinations of available types of authentication, authorization, data storage, etc. is very large, and you can also implement your own, in order to present the use of Spring Security, a certain **set of assumptions** should be made.

Application sample - `SecurityDemo`

- Based on the source: https://www.youtube.com/watch?v=TNt3GHuayXs
- App name: `SecurityDemo`
- Assumptions:
  - Users with "USER" and "ADMIN" roles:
    - Using the **default Spring Security mechanisms**, e.g. in the database, these roles will be saved **with the prefix** "ROLE_" as "ROLE_USER" and "ROLE_ADMIN"
  - MySQL user database (from XAMPP):
    - This causes the need to implement the `loadUserByUserName()` method, which **will use JPA**
  - Create sample users in the application or **directly in the database**:
    - It will **not be possible** to create users through a website
  - **Default** login pages
    - Spring Security provides controllers, actions, and views that are ready for logon and logoff. Normally available at "`/login`" and "`/logout`". Of course, you can also write your own.
  - Add role-based access rules to selected pages.
- Spring Boot 3.5.7
- Dependencies:
  - Lombok
  - Spring Web
  - Thymeleaf
  - Spring Security
  - Spring Data JPA
  - MySQL Driver

File `application.properties`

- As for application `Study`

```
spring.datasource.url=jdbc:mysql://localhost:3306/springframework
spring.datasource.driverClassName=com.mysql.cj.jdbc.Driver
spring.datasource.username=SpringRoot
spring.datasource.password=myPass1234#

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

View files

- In `resources/template`:
  - `admin.html` – Finally, it will only be available to the admin
  - `index.html` – Finally, it will be available to everyone
  - `user.html` – Finally, it will be available to a regular user and admin
- Controller:
  - `SecurityController.java`

```java
@Controller
public class SecurityController {
    @GetMapping("/")
    public String home(){
        return "index";
    }
    @GetMapping("/adm")
    public String adminPage(){
        return "admin";
    }
    @GetMapping("/user")
    public String userPage(){
        return "user";
    }
}
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Index file</title>
</head>
<body>
<h1>Page for all</h1>
<p>
  Even not logged user can read this.
</p>
</body>
</html>
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
  <title>For users</title>
</head>
<body>
<h1>Page for users.</h1>
<p>
  Admins and users can access this page.
</p>
</body>
</html>
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>For administrators</title>
</head>
<body>
<h1>Page for admins.</h1>
<p>
  Only admins can access this page.
</p>
</body>
</html>
```

Security configuration using annotations

- Od Spring Security 6 nie trzeba rozszerzać specjalnej klasy, tylko dodać adnotację `@Configuration` do dowolnej klasy, która będzie zawierać ewentualne zasady bezpieczeństwa.
  - Przy tworzeniu projektu w tym celu tworzony jest plik `SecurityConfiguration.java`
- Wstawimy tam reguły bezpieczeństwa dla endpointów jako ziarenko-metodę zwracającą obiekt klasy `SecurityFilterChain`.
- Na podobnej zasadzie w tej klasie stworzymy ziarenko-metodę, która zwraca obiekt klasy `PasswordEncoder`, który będzie służył do kodowania haseł.
- Adnotacja `@EnableWebSecurity` pozwala właśnie na użycie filtrów bezpieczeństwa, natomiast `@EnableMethodSecurity` pozwoli na adnotacje bezpieczeństwa przy akcjach kontrolerów.

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfiguration {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers(PathRequest.toStaticResources().atCommonLocations())
                                .permitAll()
                        .requestMatchers("/adm").hasRole("ADMIN")
                        .requestMatchers("/user").hasAnyRole("USER", "ADMIN")
                        .requestMatchers("/").permitAll()
                        .anyRequest().authenticated()
                )
                .formLogin(form -> form.permitAll());
        return http.build();
    }

//    @Bean
//    public PasswordEncoder passwordEncoder() {
//        return NoOpPasswordEncoder.getInstance();
//    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Access rules

- Elements of `securityFilterChain(HttpSecurity http)`:
  - `http.authorizeHttpRequests()` – all HTTP requests require authorization
  - `requestMatchers(PathRequest.toStaticResources().atCommonLocations()).permitAll()` - all requests to static resources (usually in the `resources/static` folder) **do not require** authorization (`pertmitAll()`)
  - `.requestMatchers("/adm").hasRole("ADMIN")` – if the request URL is "`/adm`", the "`ADMIN`" role is required for the request to be handled
  - `.requestMatchers("/user").hasAnyRole("USER","ADMIN")` – if the request URL is "`/user/`" to handle the request if the logged in user has the "`ADMIN`" **or** "`USER`" role
  - `.requestMatchers("/").permitAll()` – If it is a request to the main page, then (**exceptionally**) anyone can open such a page.
  - `.and()` – and there will be other rules that apply at the same time
  - `.formLogin(form -> form.permitAll())` – Add built-in login and logout pages at "`/login`" and "`/logout`"
    - the argument is a security rule, in this case anyone can log in and log out.

How passwords are stored

```
//    @Bean
//    public PasswordEncoder passwordEncoder() {
//        return NoOpPasswordEncoder.getInstance();
//    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
```

- Passwords are usually stored in encrypted form, hash-codes, etc.
- However, in this example, to make it easier to add/delete users through the database console, passwords can be stored without encryption.
- In professional applications, CRUD operations on users (including password modifications) should also be enabled, i.e. several actions in the controller and several views should be added, which would complicate the presented example

Class `MyUserDetailsService`

- To build a properly functioning `AuthenticationManager`, you need to create a bean that returns a `PasswordEncoder` (already in the code) and a service class `(@Service)` that implements the `UserDetailsService` interface. For users in the database, you'll need to implement such a class yourself.
  - the seed and service will be injected.
- The `UserDetailsService` interface has only one method: `UserDetails loadUserByUsername(String username)`. Because it "returns" the `UserDetails` interface, it will be able to work properly if you prepare a class that implements the `UserDetails`. This will be the `MyUserDetails` class.
- Below the interface `UserDetails` (interface from the Boot Security package).

```
interfejs UserDetailsService {
    public UserDetails loadUserByUsername(String username);
}
```

```
interfejs UserDetails {
    public Collection<? extends GrantedAuthority> getAuthorities();
    public String getPassword();
    public String getUsername();
    public boolean isAccountNonExpired();
    public boolean isAccountNonLocked();
    public boolean isCredentialsNonExpired();
    public boolean isEnabled();
}
```

`User` model for JPA

- The interface requires several getters, but the model in the database may be different. Let's assume that in the database we will only remember about the user:
  - ID,
  - name,
  - password,
  - whether the account is active
  - roles in the form of a string, roles separated by commas.

```java
@Entity
@Data
@NoArgsConstructor
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int id;
    private String userName;
    private String password;
    private boolean active;
    private String roles; // roles, comma "," separated
}
```

Class `MyUserDetails`

- This class will implement the interface `UserDetails`.
- In addition to the default constructor, let's create a constructor with a parameter of type `User` to convert data from the database into data expected by Spring Security
  - Some getters will always return a fixed value

```
public class MyUserDetails implements UserDetails {

    private String userName;
    private String password;
    private boolean active;
    private List<GrantedAuthority> authorities;

    public MyUserDetails(User user){
        this.userName=user.getUserName();
        this.password=user.getPassword();
        this.active=user.isActive();
        this.authorities=Arrays.stream(user.getRoles().split(","))
                .map(SimpleGrantedAuthority::new)
                .collect(Collectors.toList());
    }
    public MyUserDetails(){
    }
    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return authorities;
    }
```

```
    @Override
    public String getPassword() {
        return password;
    }
    @Override
    public String getUsername() {
        return userName;
    }
    @Override
    public boolean isAccountNonExpired() {
        return true; //changed
    }
    @Override
    public boolean isAccountNonLocked() {
        return true; //changed
    }
    @Override
    public boolean isCredentialsNonExpired() {
        return true; //changed
    }
    @Override
    public boolean isEnabled() {
        return active; //changed
    }
}
```

Class `MyUserDetails` – explanation.

```
this.authorities=Arrays.stream(user.getRoles().split(","))
        .map(SimpleGrantedAuthority::new)
        .collect(Collectors.toList());
```

- Explanations for creating authorizations (roles).
  - The `SimpleGrantedAuthority` class is an implementation of the `GrantedAuthority` interface, whose list of objects is expected in the `getAuthorities()` method.
  - From `user` data, we get the string using `getRoles()`. Then we divide it into an array of strings, where the place of division will be commas (`split(",")`), we make a stream of strings (`Arrays.stream`), which allows further **streaming operations**. Then each string is **mapped** to the constructor of the `SimpleGrantedAuthority` class, so each string will be an argument to that constructor. A stream of objects of the above class will be created. Ultimately, we need to have a list (collection) of roles, so you need to perform the `collect()` operation, which gets information about the method of converting to the `Collectors.toList()` collection.
  - This can also be done in classic way by creating an array of strings and then as loops over the elements of this array and the `add()` operation for the list of the above class.

Repository `UserRepository`

- Reminder: The `UserDetailsService` interface requires the implementation of name-based user data acquisition, so…
- … in the JPA users repository, need to retrieve data about the user based on their name, i.e. the method `findByUserName(String name)`.
  - **Reminder:** JPA will create an appropriate SQL query for us based on **the components of the method name** (camel notation)
- File: `UserRepository.java`

```java
@Repository
public interface UserRepository extends JpaRepository<User,Integer> {
    public Optional<User> findByUserName(String name);
}
```

Service `MyUserDetailsService`

- In the `MyUserDetailsService` class, inject through `@Autowired` user repository and implement the `loadUserByUsername()` method using this repository.
- If the user is not in the database, throw an exception `UsernameNotFoundException`.
- If it is, we use the `MyUserDetails` class constructor described earlier, which transforms data from the database.
- File: `MyUserDetailsService.java`

```java
@Service
public class MyUserDetailsService implements UserDetailsService {

    @Autowired
    UserRepository userRepository;
    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Optional<User> user=userRepository.findByUserName(username);
        if(user.isEmpty())
            throw new UsernameNotFoundException("Not found : "+username);
        return new MyUserDetails(user.get());
    }
}
```

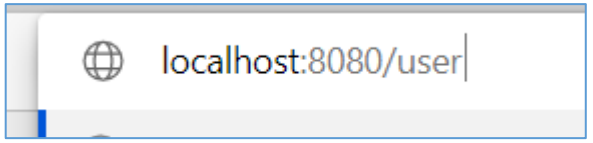
Add users to the database

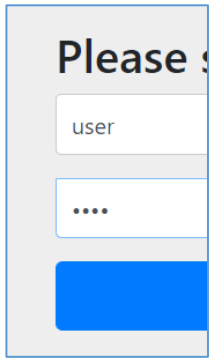
- In Spring Security, when we create roles through `SimpleGrantedAuthority`, the role name in the string must be preceded by the prefix "`ROLE_`", i.e. in this example, the "`ROLE_USER`" and "`ROLE_ADMIN`" roles will be stored in the database
- Let's create two users in a database.
  - User named "`admin`" with the same password, with the role "`ROLE_ADMIN`" (`id=0`)
  - User named "`user`" with the same password, with the role "`ROLE_USER`" (`id=1`)
- Both users are active

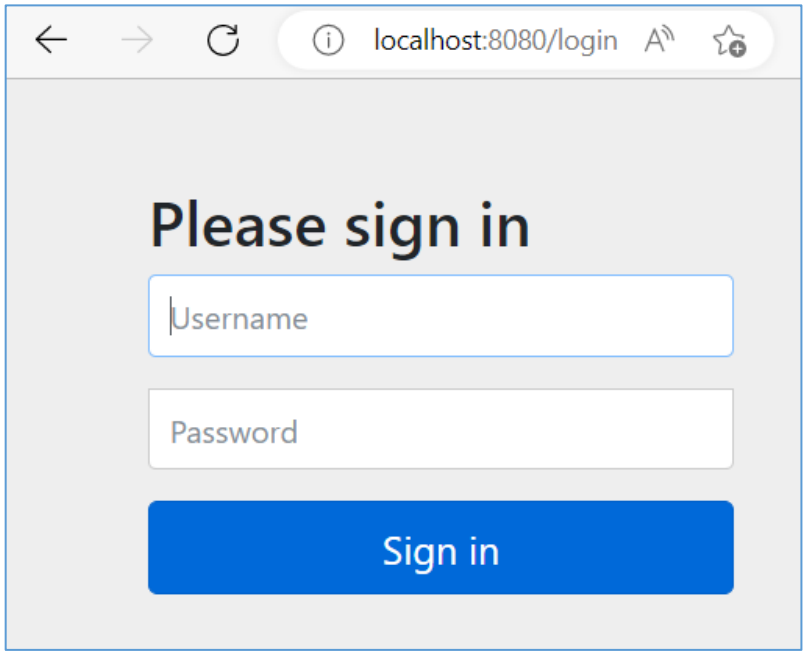
id	active	password	roles	user_name
0	1	admin	ROLE_ADMIN	admin
1	1	user	ROLE_USER	user

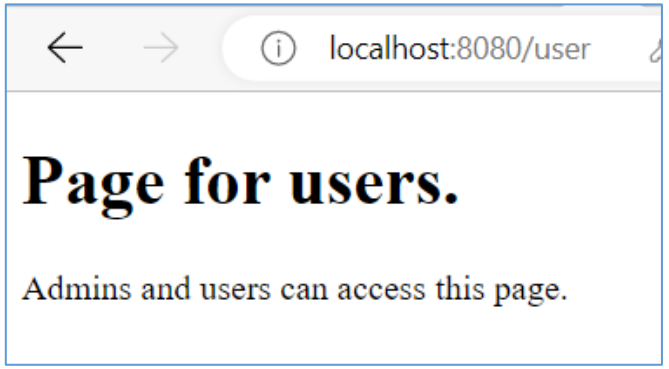
Demonstration of the application 1/3

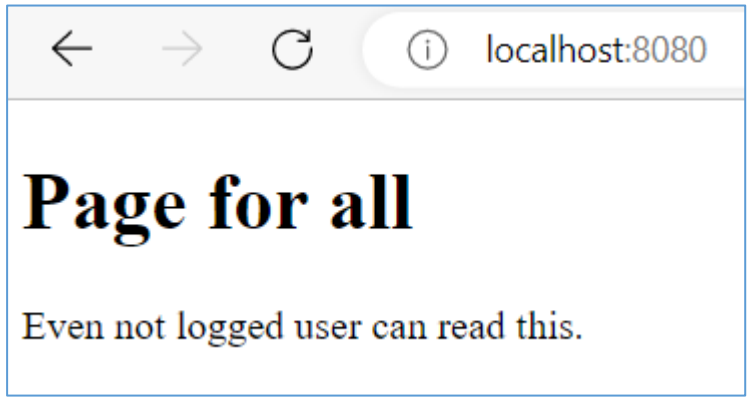
- User not logged in:
  - `/`
  - `/user`
- Log in as „`user`”
  - `/user`
  - `/`

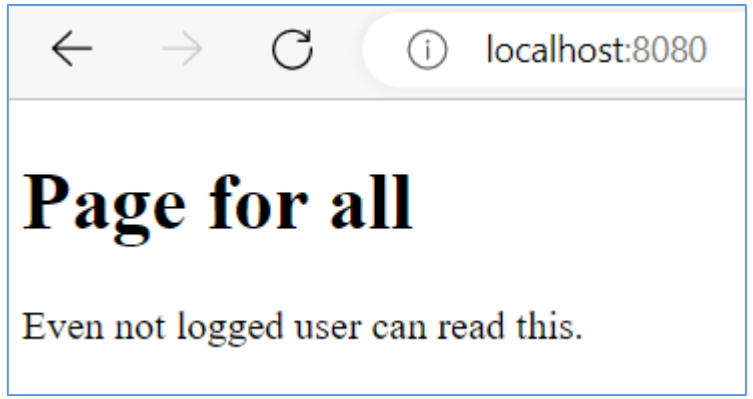






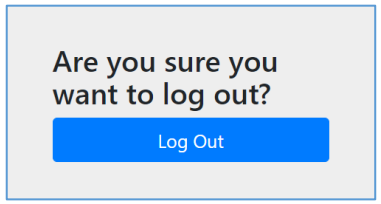


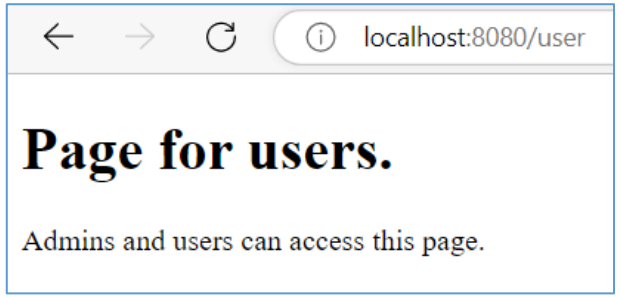


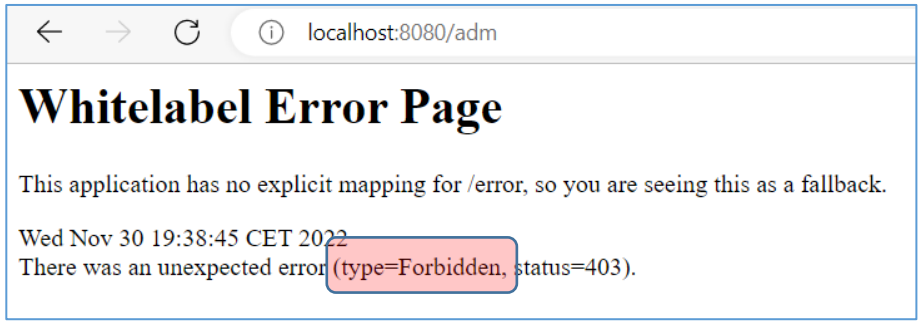


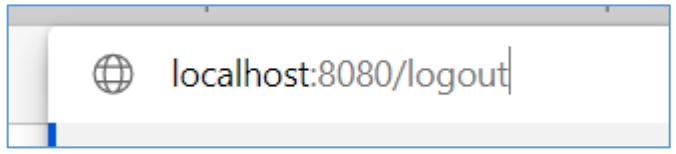
Demonstration of the application 2/3

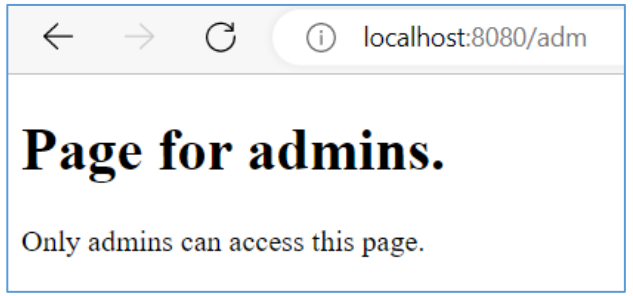
- As „`user`”:
  - `/adm`
  - `/logout`
- Log in as „`admin`”
  - `/adm`
  - `/user`
  - `/`

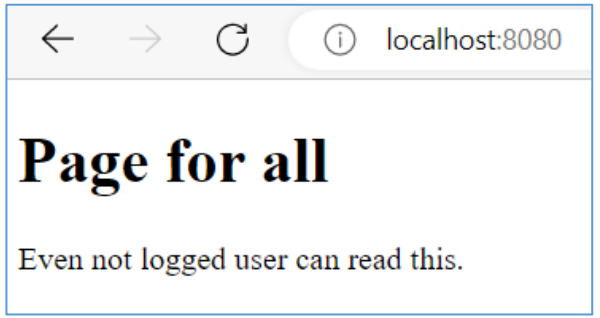






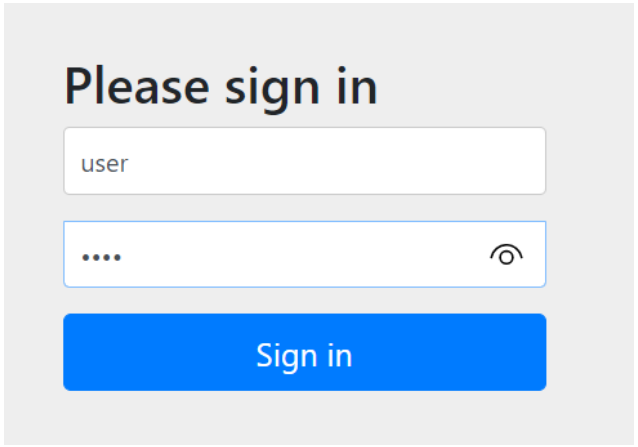


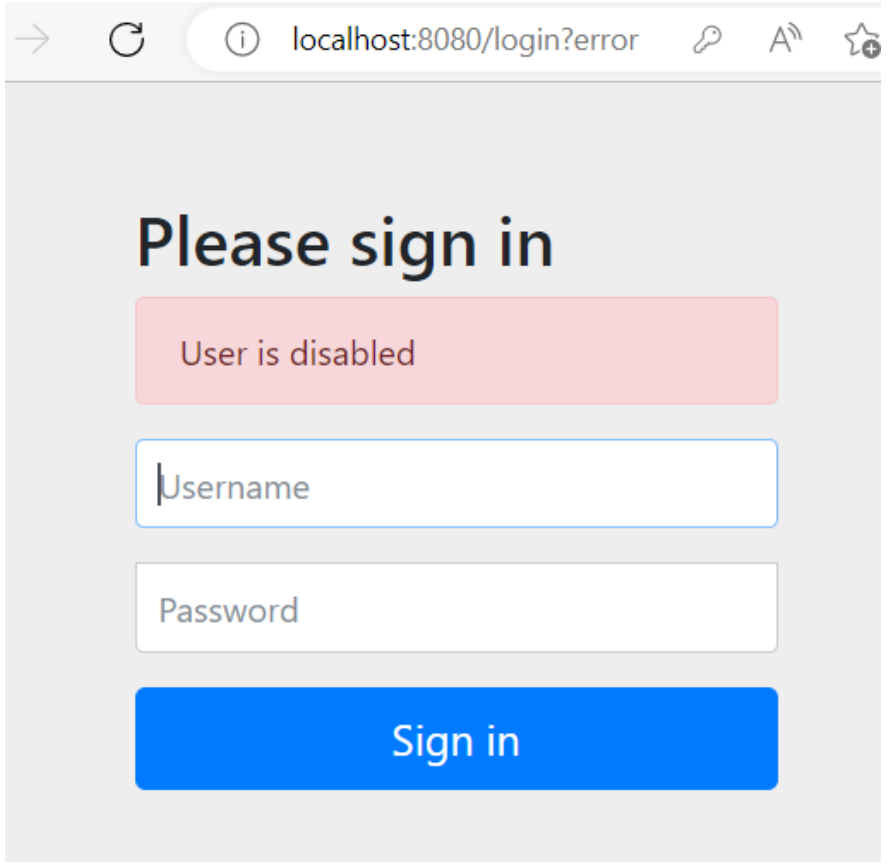




Demonstration of the application 3/3

- Logoff
- Change in user database "`user`" to **inactive**
  - `/login` – I get a message that the user is inactive





Spring Security - summary

- To start working (especially with a database) it requires the implementation of several classes.
  - However, we are guaranteed correct operation in all possible situations.
- Flexible – allows for many combinations of elements related to authorization and authentication, login/logout, user management, etc.
- Extensible – a fairly standard use of this framework has been presented, but you can also add your own mechanisms if they have not been implemented yet.
  - However, this requires understanding the operation of the filter stream and "plugging" into the right place.
- Allows the use of multiple authorization and authentication mechanisms at the same time.
- The approach using roles is presented, but it can be quite easily extended to the level of privileges that are assigned to roles, while the rules of access to pages are based on privileges.
- There are also ready-made classes to use authentication from services such as Facebook, Google, etc.
- There is the **Spring Method Security** package that allows **authorization annotations** before methods (e.g. of controller).

API Controllers

JSON format

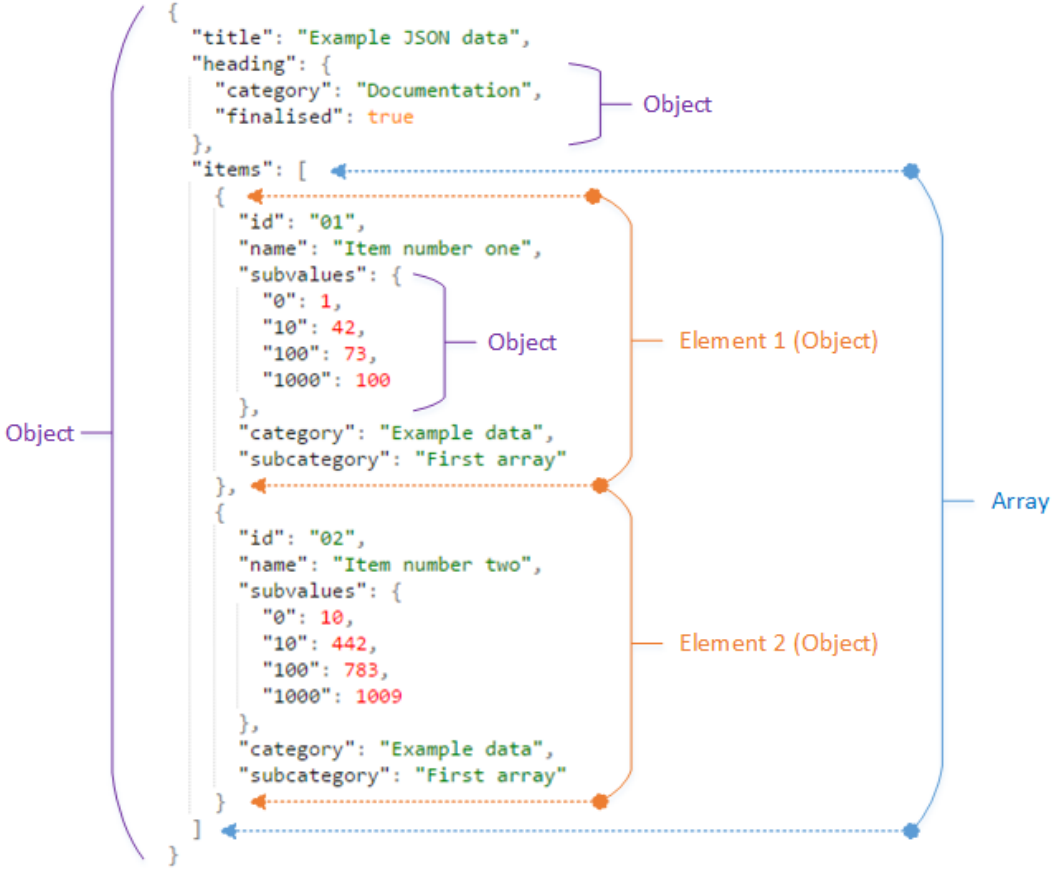
ReST and RESTful approach

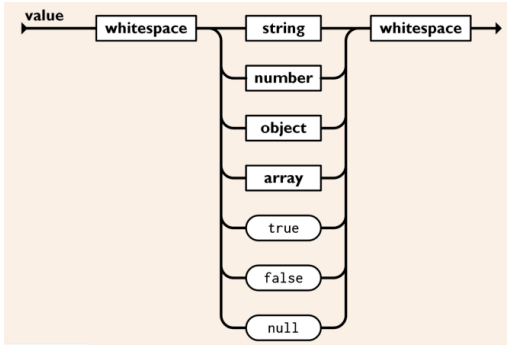
API controller for communicating with the frontend in an SPA application.

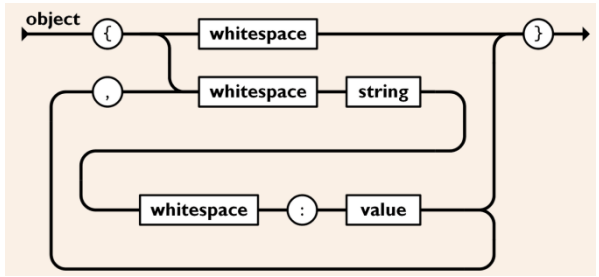
Postman - Application for testing.

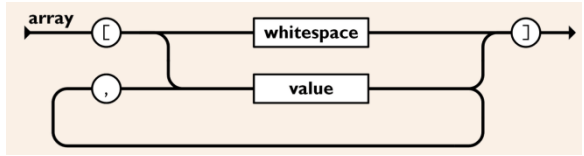
JSON format

- JSON, JavaScript Object Notation – text-based lightweight computer data exchange format
  - https://www.json.org/json-en.html
- The whole thing is one JavaScript object.
- Objects are written in curly brackets, arrays in square brackets.
- An object is a dictionary of key-value pairs separated by a colon, the next pair after a comma.
- Only very basic types are allowed (as below for value).
- Example from https://docs.exivity.com/data-pipelines/extract/parslets









ReST and RESTful approach

- REST, ReST - **Re**presentational **S**tate **T**ransfer
  - Designed around resources (not operations).
  - URI (Uniform Resource Identifier) defining a specific resource (see next slide).
  - The client communicates with the server by exchanging data representation (JSON).
  - A unified interface using verbs from HTTP methods: GET, POST, PUT, PATCH and DELETE.
  - Statelessness as in HTTP.
  - It is driven by hyperlinks in the representation (below).
- There are 4 levels of REST API implementation:
  - 1) define one URI and use POST requests to that URI for everything.
  - 2) Create a separate URI for each resource.
  - 3) Use HTTP methods to define resource operations.
  - 4) use HATEOAS (Hypertext as the Engine of Application State) hypermedia.
- RESTful means using the 4th level.

```
{
    "account": {
        "account_number": 12345,
        "balance": {
            "currency": "usd",
            "value": 100.00
        },
        "links": {
            "deposits": "/accounts/12345/deposits",
            "withdrawals": "/accounts/12345/withdrawals",
            "transfers": "/accounts/12345/transfers",
            "close-requests": "/accounts/12345/close-requests"
        }
    }
}
```

HTTP methods and RESTful addresses

HTTP Method	URL address	Description
GET	`/api/student`	Retrieving all objects `Student`
GET	`/api/student/1`	Retrieving the `Student` object with `id` equal to 1
POST	`/api/student`	Creation of a new `Student` object
PUT	`/api/student/1`	Replacing an existing object
~~PATCH~~	~~`/api/student/1`~~	~~Modification of the `Student` object with `id` equal to 1~~
DELETE	`/api/student/1`	Deleting the `Student` object with `id` equal to 1

- General rule of addressing
- At the beginning: `/api/` or `/api/v1/`, `api/v2/` etc.
- Then the name of the resource or set of functionalities:
  - Divided by '/' characters: `studies/groups/enrollments`,
- Possible obvious parameters: `/{id}`
- Possible optional or additional parameters as query string:
  - `?pos_from={from}&pos_to={to}`
- The PATCH method is rarely implemented, it has a rather complicated call (describing the sequence of changes)

Payload in API requests

- Payload is the content of the body in the request and response.
- It can be in different formats:
  - JSON will be used in the lecture.
  - It used to be XML, but it's too verbose.
- The presence of charge follows quite logically from the operation:
  - GET – none in the request, one or all resources in **response**.
  - POST – in the **request**: minimum required resource fields, in **response**: the entire created resource from the data store (possibly no load).
  - PUT: in **request**: resource to create/replace, **response** as in POST.
  - ~~PATCH: in the **request**: fields requiring changes, in the response – no. Only return code for the confirmation or no changes~~.
  - DELETE: Missing payload in request and response.

Creating an API Controller

- For the API controllers themselves, we only need one dependency:
  - Spring Web
- Controllers communicate (most often) using data streams stored in JSON format, so the controller doesn't need, for example, Thymeleaf.
- The API controller class should have the `@RestController` annotation.
  - which combines `@Controller` and `@ResponseBody`.
- Practically always, the API controller expects **a service** to perform operations on the data collection.
  - It's best to **inject** such a service **in** the controller's **constructor**.

- For this demonstration, we will use (and extend) the `StudentListH2Lombok` application presented in the previous lectures, as it already has a service implemented to manage the student database.
  - This means that data will be accessed both through web pages and the API controller.

Project StudentListApiController

- Code copy from the StudentListH2Lombok project.
- Dependencies:
  - **Spring Web**
  - Thymeleaf
  - Spring Data JPA – Provides ORM mapping
  - H2 Database – Fast database in memory or file
  - Lombok – Annotation tool that simplifies writing standard code
- Adding an API controller.
- Testing the API controller.

API Controller - code 1/2

- `@RestController` annotation.
- Returning objects (which will be converted to JSON format).

```java
@RestController
@RequestMapping("/api/student") // Base URL for all methods
public class StudentApiController {
    private final StudentService studentService;

    // Service injection via constructor
    public StudentApiController(StudentService studentService) {
        this.studentService = studentService;
    }

    // 1. READ (Download all) - GET /api/student
    @GetMapping
    public List<Student> getAllStudents() {
        return studentService.getAllStudent();
    }

    // 2. READ (Download by ID) - GET /api/student/{id}
    @GetMapping("/{id}")
    public Student getStudentById(@PathVariable Long id) {
        // Typically error handling is added here, e.g. returning 404 if the student does not exist
        return studentService.getStudentById(id);
    }
```

API Controller - code 2/2

- using different HTTP methods

```
    // 3. CREATE - POST /api/student
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED) // Returns status 201 Created
    public Student createStudent(@RequestBody Student student) {
         return studentService.addStudent(student);
    }

    // 4. UPDATE (by ID) - PUT /api/student/{id}
    @PutMapping("/{id}")
    public Student updateStudent(@PathVariable Long id, @RequestBody Student studentDetails) {
        // The implementation should find the student by ID, update it based on studentDetails
        // and save it.
        studentService.deleteStudentById(id);
        return studentService.updateStudent(studentDetails);
    }

    // 5. DELETE (by ID)- DELETE /api/student/{id}
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT) // Returns status 204 No Content
    public void deleteStudent(@PathVariable Long id) {
        studentService.deleteStudentById(id);
    }
}
```

Improved `StudentService` class and `StudentRepository` interface

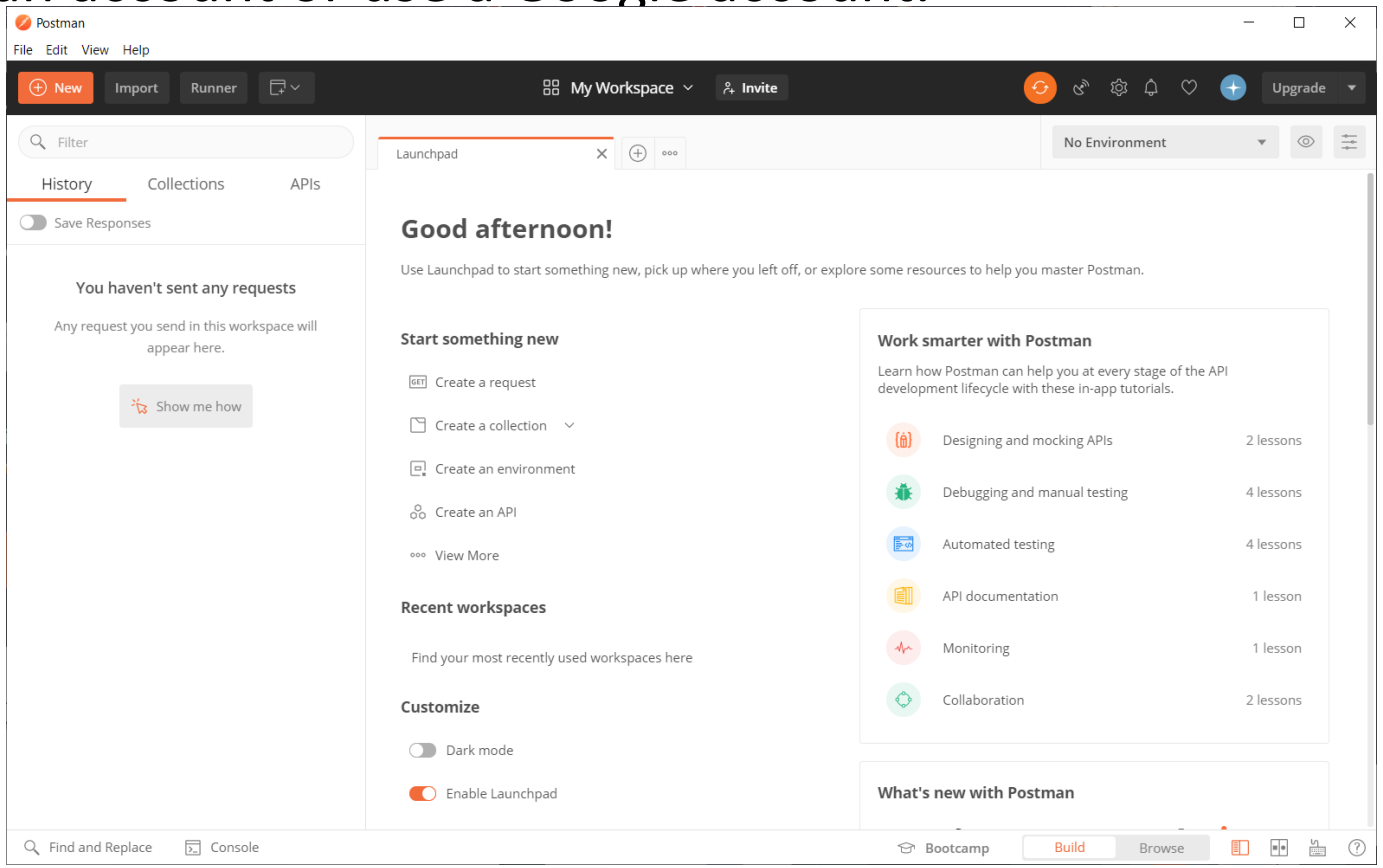
- The `addStudent()` method should always create a new student with an unique `id` (i.e., it ignores the `id` in the parameter).
- The `findTopId()` method has been added.

```java
public interface StudentRepository extends JpaRepository<Student,Long> {
        ...
    @Query("select max(s.id) from Student s")
    Optional<Long> findTopId();
        ...
}
```

```java
@Service
public class StudentService {
        ...
    // add with Max(id) + 1
    public Student addStudent(Student student) {
        Optional<Long> maxId=studentRepository.findTopId();
        if (maxId.isPresent()) {
            student.setId(maxId.get()+1);
        }
        else
            student.setId(1);
        return studentRepository.save(student);
    }
        ...
}
```

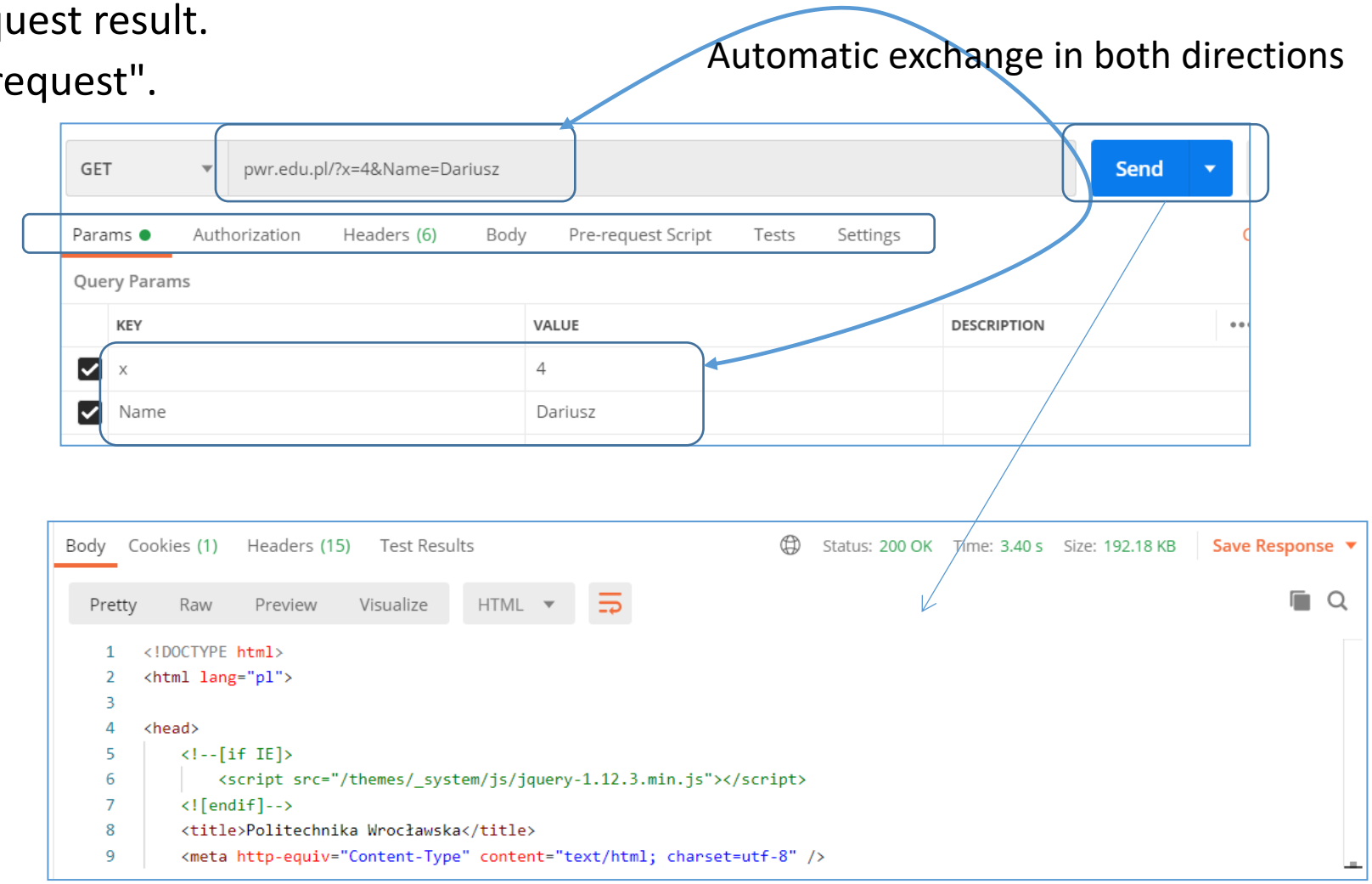
Postman – testing tool

- https://www.postman.com/downloads/
  - Windows.
  - MacOS.
  - Linux.
- You must create an account or use a Google account.

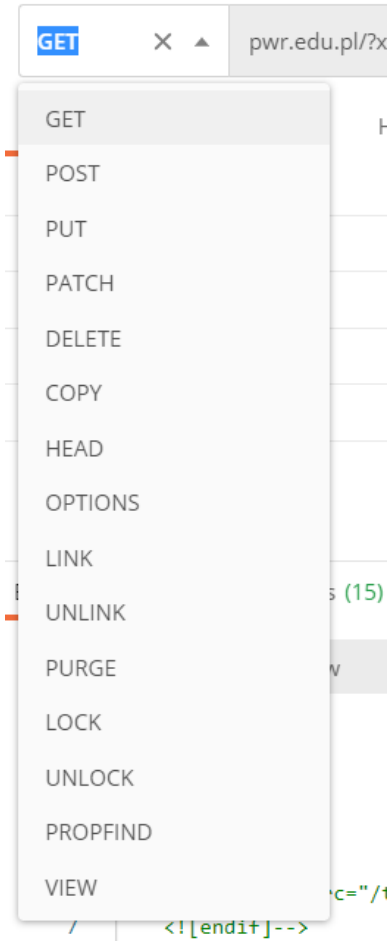


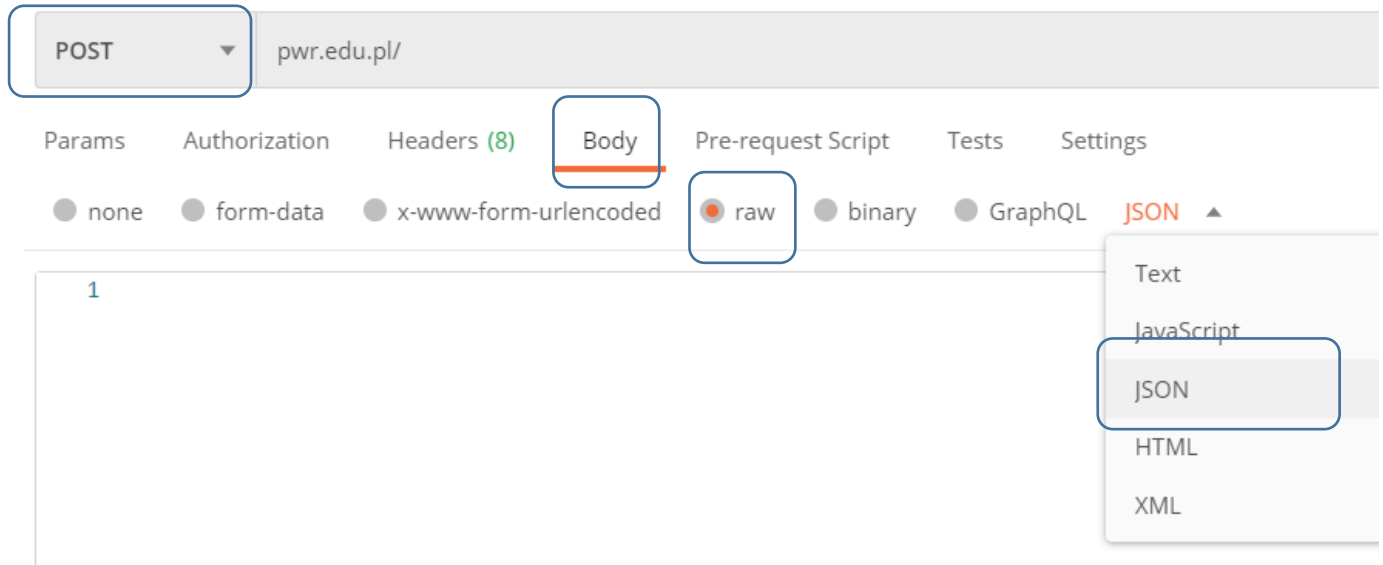
Postman – use 1/2

- Sending an HTTP request in the selected format.
- Receiving the request result.
- Select "Create a request".



Postman – use 2/2

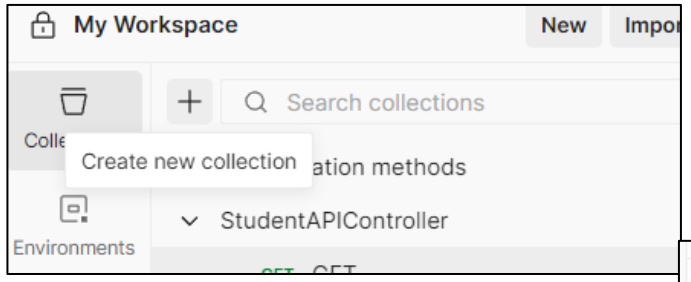


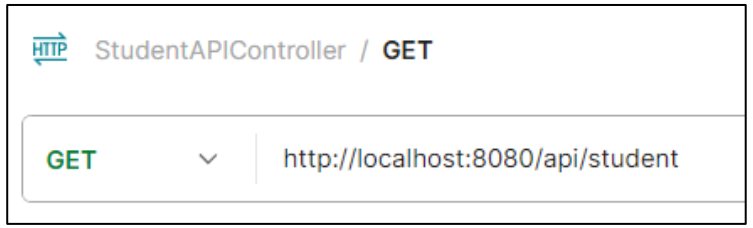


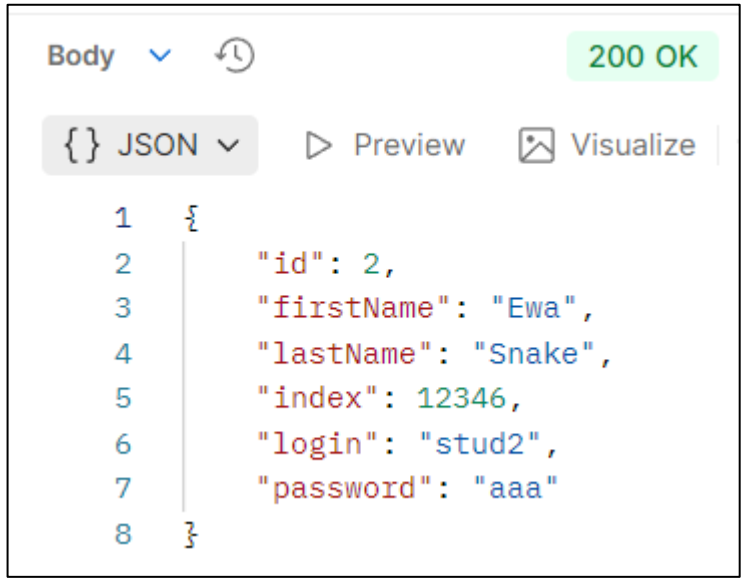
- The tool is well suited for testing controllers written in RESTful using the JSON format
  - Suggests and checks JSON syntax
  - Other formats can also be used
  - Creates test-cases and test-suites

Testing with Postman 1/3

- Creating the `StudentAPIController` collection.
- Adding requests and running them.

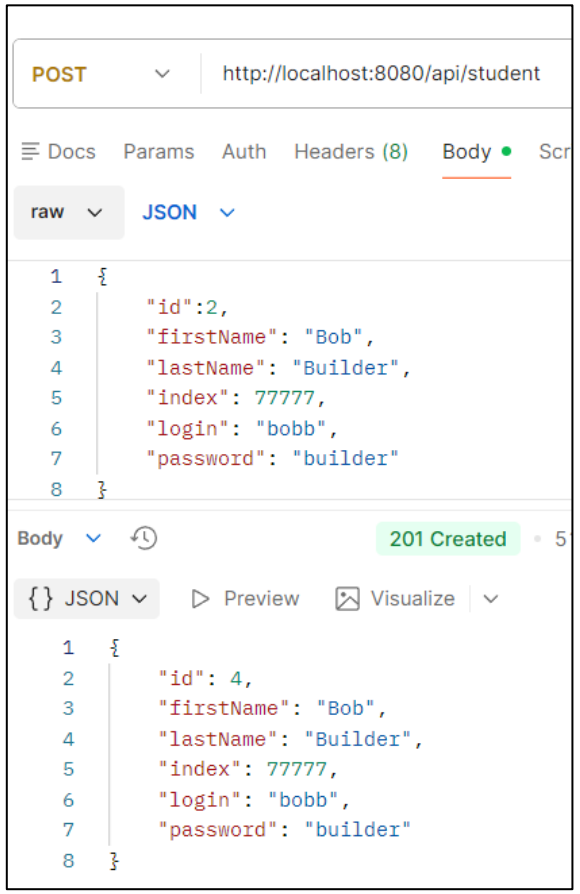


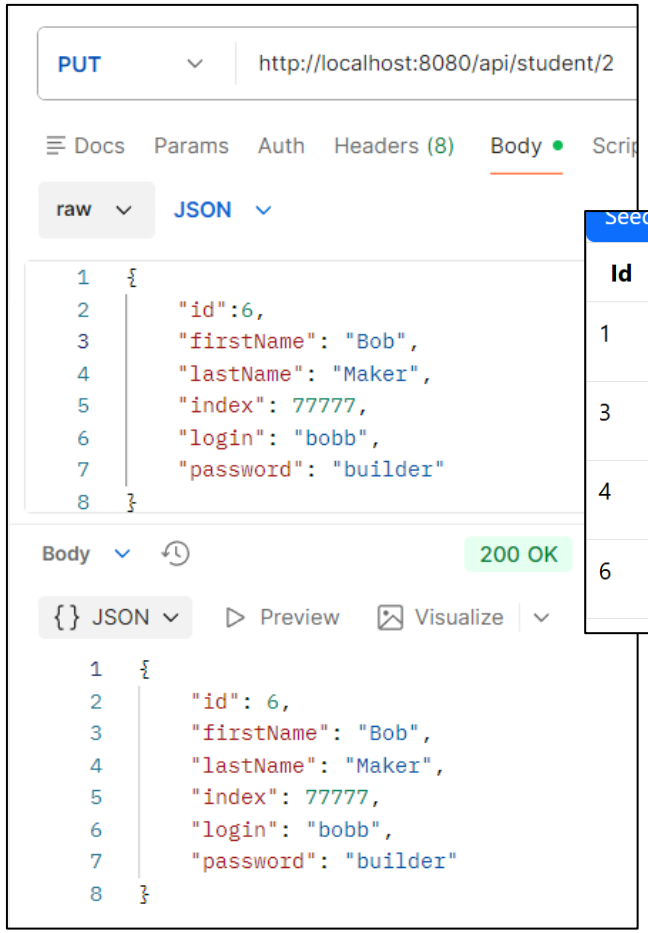


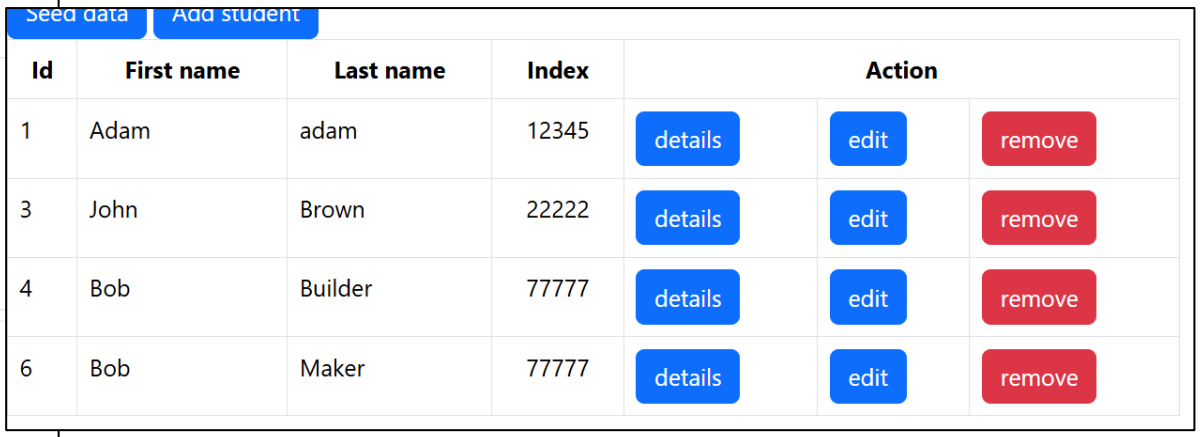


Testing with Postman 2/3

- The POST method ignores the input id=2, and the new object has id=4.
- The PUT method replaces the data in the object with id=2, as can be seen in the table.

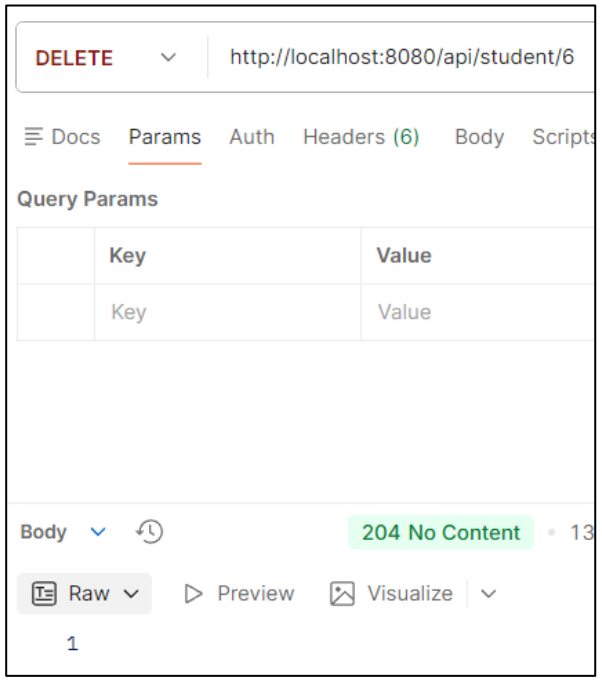


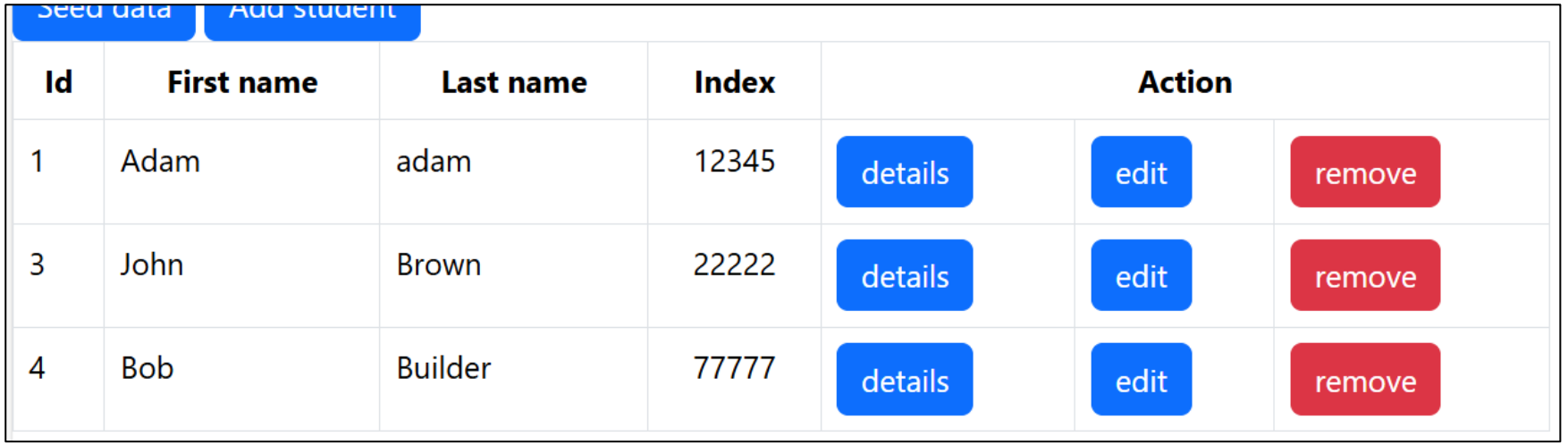




Testing with Postman 3/3

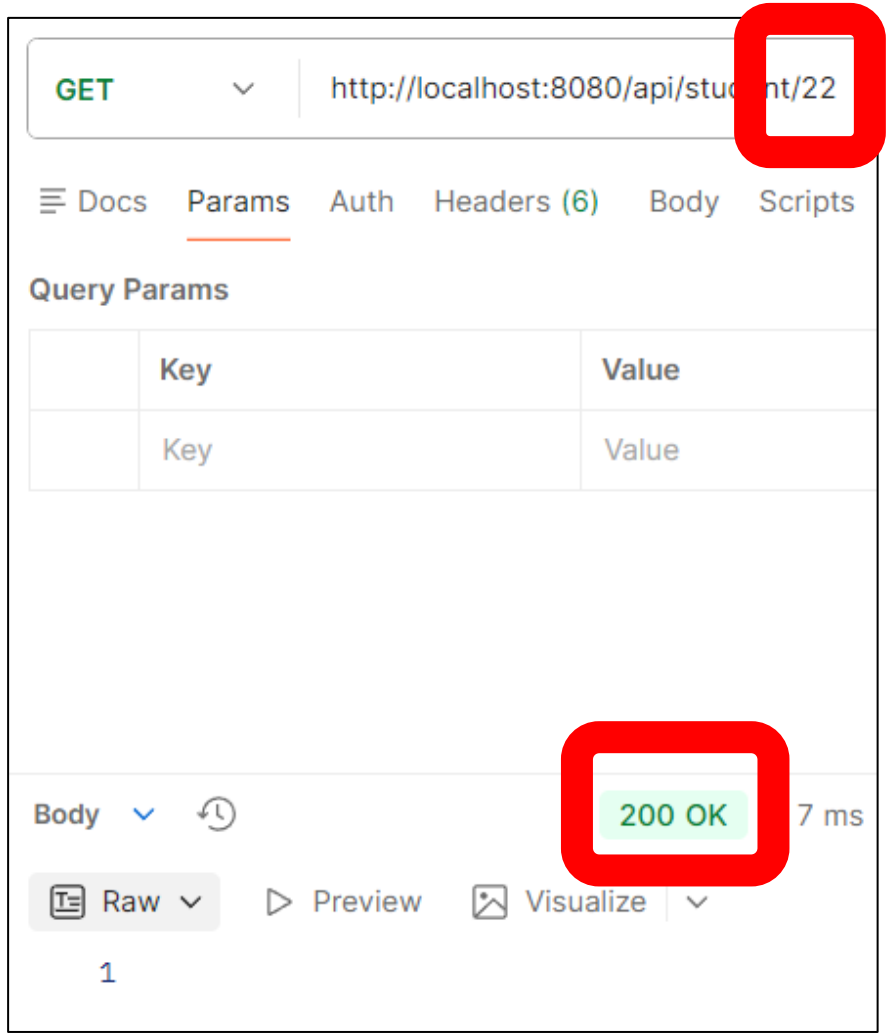
- Deleting object with id=6.





Problem of missing student with given ID

- Most controller actions encounter the problem of a missing student with the specified ID. Detecting this situation should result in an HTTP response with the code `404 NOT FOUND` (not `200 OK`).
- One solution is to return `ResponseEntity<Student>` instead of `Student`.
  - This class also remembers the response code, header data, and the response body.
- Therefore, the sample code for the GET method should look like the one on the next slide.



Modified `getStudentById()` action

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
    Student retStudent=studentService.getStudentById(id);
    if(retStudent==null) {
        return new ResponseEntity<>(HttpStatus.NOT_FOUND); // 404
    }
    else
        return ResponseEntity.ok(retStudent); // 200
}
```

- The `ResponseEntity` class has static methods for classic situations like the one above (referencing an object or **`null`**). This allows for an even shorter write (using the `ofNullable()` method), as shown in the box below.

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
    return ResponseEntity.ofNullable(studentService.getStudentById(id));
}
```

An even better version of the action

- Currently, it's recommended NOT to use **`null`** values in programming. Instead, the `Optional<T>` class is used as the type. This type appears in the `findById()` method in `JpaRepository<>`. It's also worth using it in services.
- The service would then have a `studentService.getStudentById(id)` method, returning an `Optional<Student>` instead of a `Student`.
- For such situations, the `ResponseEntity` class has a static `of(value)` method, and the final code would look like this:

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudentById(@PathVariable Long id) {
    return ResponseEntity.of(studentService.getStudentById(id));
}
```

Testing new versions

- Any of the previous codes now generate the same response for a GET request with id.